Proofs and Prices: Machine-Checked Soundness and Measured Trade-offs in a Minimalist Chess Engine

Thomas Dybdahl Ahle Second Author

Draft of August 9, 2026

Abstract

Sunfish is a chess engine whose whole logic fits in 147 lines of Python, and whose docstrings state, in unusually precise English, the contracts its search is supposed to satisfy. We report on a campaign that treated those docstrings as latent formal specifications: we transcribed them into Lean 4, proved the ones that are true, refuted the ones that are not, named the ones that are only conditionally true, and — where a condition turned out to be both load-bearing and *false* — changed the engine so that it verifies at run time what it had been assuming. In parallel, every resulting engine change was priced with ELO tournaments under a fixed empirical methodology. The formal deliverables include a sorry-free proof of the engine's core fail-soft null-window contract; a machine-checked seven-position counterexample showing the original stalemate correction is sound only under a named hypothesis; a proof of a maintainer-conjectured invariant about killer moves, later strengthened to a full lifecycle theorem; and a machine-checked transposition-table inconsistency induced by gamma-adaptive Late Move Reductions. For the latter we first proved an interval-shaped statement — fail highs bound one value function, fail lows another, the truth in between — and then deleted it: such a claim is a guarantee only if the gap between the two functions is bounded, and no bound on search instability is provable (one ply of depth can hide a mate), so the interval reading was never a specification, only a description of a bug. The only contract the engine recognizes is the *point spec*: every stored bound describes one value function determined by the transposition key. The campaign's second half is a single mistake found three times. Sunfish repeatedly inferred *legality* from a *score*: a fail-soft report below a sentinel was read as "this node has no legal moves". Three such inferences were refuted on real boards, each with a witness position and a real contradictory table entry. The repair is architectural — *verify-on-suspicion*: the move loop carries a cheap legality certificate alongside the value fold, and pays for a direct board-predicate proof only when a bound would otherwise cross a terminal value without one. With it, the invariant that a king-capturable node reports exactly the mate sentinel — refuted for the old loop — is restored *by construction* and proved for the shipped consumer, which is in turn proved equal to an executable reference specification. The final premise to fall was a chess statement ("you cannot be in zugzwang while delivering forced mate"), which we could not assume, because a counterexample search found it to be false; the engine now settles the question with one search probe at a window where both failure directions are decisive. The result is a correctness layer — the search's bracketing property and its table's non-contradiction — that carries *no chess assumption at all*; zugzwang survives only in a second layer that governs accuracy and that the table can never depend on. Each remaining assumption is a named hypothesis in a ledger of bets and, where possible, measured on the production engine, which plays publicly on Lichess. Proofs identify the trade; tournaments price it; and when the trade is a false premise, neither loop is enough — the engine has to be changed.

1 Introduction

Sunfish[1] is a minimalist chess engine: with comments and whitespace removed, the engine proper is 147 lines of Python, yet it plays at ratings above 2000 on Lichess, where it runs in production as the bot `sunfish-engine`. What makes it an unusual verification target is not its size but its *documentation*: the docstring of its central routine, `Searcher.bound`, is a complete two-sided fail-soft contract, stated a decade before anyone tried to prove it. The comments scattered through the search — "`lower <= s(pos) <= upper`", "the opponent will for sure just stand pat", "sunfish may report 'mate', but then after more search realize it's not a mate after all" — turn out to be, respectively: a table invariant, a dischargeable lemma, and the exact failure mode of a hypothesis that a machine-checked counterexample shows to be necessary. The docstrings were latent formal specs.

This paper reports a campaign, conducted over five days on the live repository, that ran two loops in parallel:

- **The proof loop.** Every search feature was transcribed into a small, dependency-free Lean 4 model (core Lean plus the `omega` tactic; no Mathlib; a full build takes seconds) and either proven sorry-free, refuted by an explicit machine-checked counterexample, or reduced to a *named hypothesis* whose statement — not whose proof — is the deliverable.
- **The price loop.** Every engine change motivated by the proofs (and every change the proofs advised against) was priced by an ELO tournament between frozen engine builds, under the methodology of Section 14, on the same engine that plays real games on Lichess.

The two loops answer different questions and neither substitutes for the other. A proof tells you *what* a pruning trick costs in soundness — which invariant it weakens, at which positions, under which window. A tournament tells you what that weakening costs (or earns) *in ELO*. Several of our measured numbers (Table 5) show soundness restorations that are strictly ELO-negative, and unsound-in-the-point-sense optimizations that are strongly ELO-positive; the formal work is what lets us merge the latter with our eyes open, with a two-line containment for the specific inconsistency the proof predicts — and, once the interval reading of that containment was recognized as guaranteeing nothing, to retire the optimization deliberately at a measured price (Section 7.4) — and finally, when a last measurement campaign showed that the honest form of the optimization is worth exactly zero, to delete the mechanism outright, formal apparatus and all (Section 7.5).

A third loop appeared once the campaign was under way, and it is the one this draft is mostly about. Some named hypotheses are not bets worth keeping: they are *false*, and load-bearing. When that happens neither proving nor pricing helps — the engine has to be changed so that it stops needing the assumption. Three times in this campaign a hypothesis of the form "a score below this sentinel means no legal move exists" was refuted on a real board, with a real contradictory transposition entry to show for it (Section 8); the response was an architectural change, *verify-on-suspicion* (Section 9), that replaces the inference with a cheap certificate and a direct check.

Contributions. (1) A sorry-free Lean 4 proof that sunfish's fail-soft null-window search satisfies its own docstring (Section 4) — to our knowledge the first Lean 4 formalization of

[1] https://github.com/thomasahle/sunfish

null-window/MTD-bi search. (2) A faithful model of the engine's king-capture convention and stalemate correction, with a proof under four individually necessary hypotheses and a machine-checked counterexample showing the key hypothesis cannot be dropped (Section 5). (3) A soundness taxonomy for selective search (Section 7), with futility pruning and Late Move Reductions as the canonical citizens of its two nontrivial classes, and the doctrine it produced: there is no interval spec. (4) *A score is not a legality proof.* Three refuted "score implies legality" assumptions, each with a real board witness and a real crossed table entry (Section 8); the verify-on-suspicion architecture that removes them (Section 9); and the proof that the shipped consumer computes exactly the same function as an executable reference specification (`production_eq_reference`). (5) A correctness layer with *no chess premises*: the search's bracketing property against its own declared value function, and the non-contradiction of its transposition table, hold unconditionally — including the last chess statement, which was discharged by a run-time band-edge probe after a counterexample search showed it was false (Section 10). (6) An explicit invariant catalogue: what the engine guarantees, by what mechanism each guarantee is earned, and how they compose into the top-level statement, with human-readable proof sketches (Section 11). (7) The ledger-of-bets discipline (Section 13), the model–code drift guard (Section 12), and the empirical methodology with honest numbers, including artifacts we initially believed (Section 14).

Reading guide. Section 2 defines, once and explicitly, the model in which every theorem of this paper is stated — what is a Lean definition, what is a named hypothesis, what is not modeled at all — so that each later "Theorem" can be read with its exact scope in mind. Section 3 then inventories every machine-checked result: Table 1 lists the proved theorems with their Lean names and files, and Table 2 lists the machine-checked *counterexamples*, which are deliverables of equal rank. Sections 4–7 tell the story of the early results in the order they were found; Sections 8–10 tell the story of the repair. A reader who wants only the current state of the engine — what is guaranteed and why — can read Section 11 alone; it is self-contained and is the paper's answer to "what, exactly, is proved about the code that ships?"

2 The Model: What the Theorems Quantify Over

Every result in this paper is a theorem about a small Lean 4 development (`formal/` in the sunfish repository: eight files, no dependencies beyond core Lean and the `omega` tactic), not about the Python text of `sunfish.py`. This section fixes the model once, so that each later theorem can be read with its exact scope in mind; how the model–code gap is managed is discussed in Section 16, and Figure 1 shows the correspondence at a glance.

2.1 Abstract games, king-capture legality, and the true value

The base structure is chess-free:

Definition 2.1 (Game)**.** A *game* G consists of a type of positions Pos, a successor function moves : Pos $\to$ List Pos, and a static evaluation eval : Pos $\to \mathbb{Z}$. The empty move list means *the side to move has lost.* (`Game`, `GameTree.lean`)

Sunfish is a *king-capture* engine: it generates pseudo-legal moves, and legality is enforced one ply late by making king capture decisive. The model adopts this convention wholesale: a

position with $\text{eval}(p) \leq -M_{\text{L}}$ has already lost its king, and every search in the development normalizes such positions to the exact sentinel $-M_{\text{U}}$. "Check", "mate" and "stalemate" are therefore not primitives of the model; they are emergent, exactly as in the engine (Section 5). Features that need more structure extend Game conservatively: a *NullGame* adds a pass move $\text{pass} : \text{Pos} \to \text{Pos}$ (the model of `pos.rotate(nullmove=True)`); a *ValGame* adds per-move values val together with sunfish's incremental-score identity $\text{eval}(m) = -(\text{eval}(p) + \text{val}(p, m))$ as a structural field (Section 7); killer- and transposition-table models additionally assume decidable equality of positions, for table keying.

The "true score" s^* of the docstring is *defined* to be depth-limited negamax: $\text{negamax}_d(p) = \text{eval}(p)$ at $d = 0$, otherwise the maximum of $-\text{negamax}_{d-1}(m)$ over $m \in \text{moves}(p)$, folded from $LOSS = -M_{\text{U}}$ (`negamax`, `GameTree.lean`). Variants of the value function — negamaxDraw (draw-aware, Section 5) and nullValue (null-move- and repetition-augmented, Section 13.1) — are likewise definitions, not axioms. (Two further variants have been *deleted* from the development along with the mechanisms they specified: the $(V^{\text{hi}}, V^{\text{lo}})$ pair once defined for re-search LMR — Section 7 tells that story and states the doctrine behind the deletion — and negamaxDet, the per-move-reduced-depth value of the deterministic-LMR era, deleted when all reductions were removed at commit `7fdd741`, Section 7.5.)

2.2 The searches are Lean functions, one per feature

`Searcher.bound` is modeled as a total recursive Lean function mirroring the Python control flow, and each search feature is modeled in its own file as a variant of that function: the bare fail-soft skeleton bound (`bound`, `Bound.lean`); table threading (boundTT), futility (boundFut) and mate-score softening in `Tricks.lean`; the stalemate correction (boundStale, `Stalemate.lean`); the killer table as state (boundKill, `Killer.lean`); and the search with null move, repetition check, $(\text{pos}, \text{depth})$-keyed table and IID (boundNullTT, `CanNull.lean`). Two later files complete the picture: `EvalBounds.lean`, which discharges numeric hypotheses by `decide` over the transcribed piece-square tables (Section 11), and `Driver.lean`, which models the MTD-bi bisection itself and proves which windows it can actually probe (Section 4.2). The two consumers of the current design — an executable reference specification (boundD2) and the shipped loop (boundKCX) — are both in `Stalemate.lean`, together with the theorem that they compute the same function (Section 9). Two further models existed while the mechanisms they specified shipped — deterministic LMR (boundLmrDet, `LmrDet.lean`) and the historical re-search LMR (boundLmr, `Lmr.lean`) — and were deleted with those mechanisms at commit `7fdd741` (Section 7.5); git is the archive. Studying features in separate models is itself a modeling decision, and where features could interact the development proved they do not: while the reductions existed, futility ($\text{depth} \leq 1$) and LMR ($\text{depth} \geq 3$) never co-occurred (`futilityLmrDisjoint`) and reductions never touched king captures (`RedRespectsCaptures`) — both theorems proven in `Lmr.lean` and now in git history with it — which is what let the killer and stalemate arguments survive under LMR while it shipped.

2.3 Nothing is axiomatized; conditions are named hypotheses

The development contains no `axiom` declarations and no `sorry`. Everything above is a Lean *definition*; every theorem is proven from those definitions, and `#print axioms` on the main theorems reports only `propext` and `Quot.sound` (the ambient axioms of Lean's kernel). Where a claim about the engine is only *conditionally* true, the condition is not assumed globally:

it is a *named hypothesis carried in the theorem statement* — e.g. *Bounded* (evaluations lie in $[-M_U, M_U]$), *CheckProbeOK*, *MateValuesAreKingCaptures*, *NullBetOK*, *FutilityMateOK* — and Section 13 maintains the full ledger. Machine-checked counterexamples are concrete finite games (three to seven positions) whose search values are computed by Lean's kernel via `decide`; they are theorems of the form "this universally quantified statement is false", not informal counterarguments.

2.4 What is *not* modeled

Theorems quantify over *all* games of Definition 2.1; sunfish's chess is one intended instance, but board representation and move generation are entirely abstract.

The repository enforces a standing rule — *the model must always do what the code does* — so a divergence between the two is treated as a defect to be fixed in the same commit, not as a footnote. What survives that rule is a short, closed list of deliberate *scoped abstractions*: places where the model draws a layer boundary rather than making a claim about the shipped code that is false. The list is maintained in `formal/README.md` and reproduced here in full; it is the honest answer to "what is *not* verified?".

Abstraction	**Scope, and why the boundary is where it is**
QS-as-eval at the leaf	Depth 0 returns the quiescence value; QS's own stand-pat/capture recursion is that leaf's fixpoint rather than being unrolled. The boundary is sound at the point of use: the recursion only ever reaches a depth-0 child at a *non-futile* position, where the leaf is exact (Section 9.2), and the depth-0 contract itself is proven (`qsStrat_failHigh_of_capture`).
Deadline / `Stop`	Raises at node *entry*, before any store: an abort can leave a search unfinished, never a table entry unjustified.
Eviction (`TABLE_SIZE`)	Only forgets entries, which preserves every invariant stated here — and the killer invariant is proven stable under it explicitly (`killerInv_trace`).
Move ordering	Modeled through its load-bearing consequences only: `CaptureFirst` (itself discharged from the sort spec, Section 11) and the sort-equivalence of the QS break and the futility break with filters.
Transposition table	`CanNull.lean`'s layer; the theorems of `Stalemate.lean` are about the value function that every entry describes.
`Position` internals	`gen_moves` / `rotate` / `move` / `value` are axiomatized by their structural properties (the score identity, the `EvalBounds` margins). "These implement chess" is the one genuinely open item, and it is assigned to a separate track (Section 16).

Two consequences are worth stating bluntly. The theorems cannot certify that `gen_moves` implements the rules of chess — only how the search treats whatever moves it is given. And the model–code correspondence itself is established by audit plus an automated drift guard (Section 12), not by machine proof: that bridge is the largest remaining piece of trust in the construction, and we say so again in Section 16.

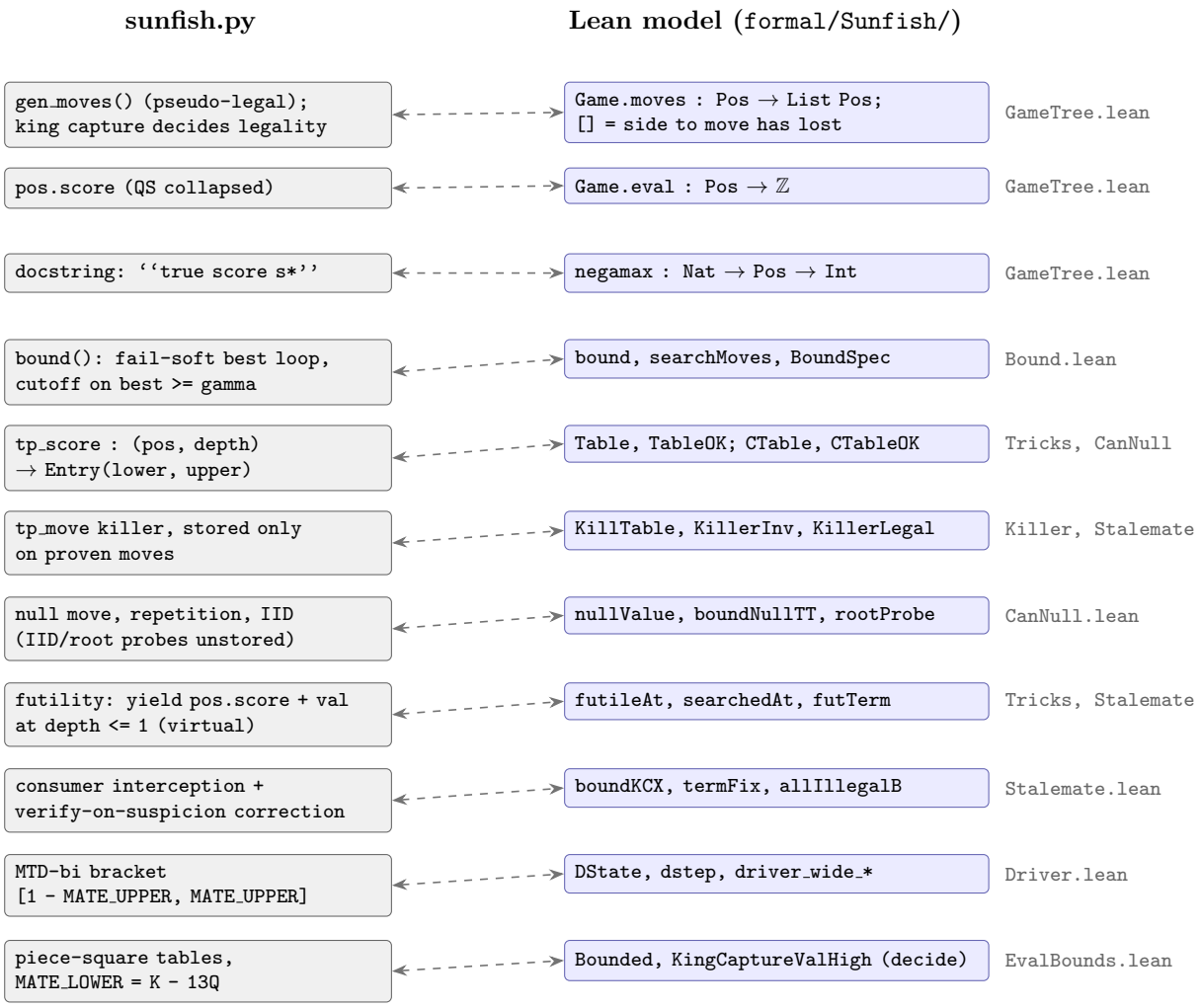

Figure 1: The model–code correspondence, feature by feature. Left: the mechanism in `sunfish.py`; right: the Lean definition that transcribes it, with its file. The map is re-audited at every commit that touches an audited region — most recently after the removal of all reductions (`7fdd741`), the removal of the `can_null` flag (`eda66ee`), the verify-on-suspicion landing (Section 9) and the band-edge arm (Section 10) — and the full line-by-line map is the correspondence table in `formal/README.md`, mechanically protected by the drift guard of Section 12. Everything on the right is a *definition*; the theorems of Table 1 are proven about these definitions with no axioms beyond Lean's kernel.

3 What Is Proved: The Theorem Inventory

Tables 1 and 2 inventory every machine-checked result cited in this paper, by Lean name and file. The separation matters: Table 1 lists *proved theorems* — positive soundness statements, unconditional unless a named hypothesis is listed — while Table 2 lists *machine-checked counterexamples*: kernel-checked refutations of statements one might have hoped were theorems. Both build sorry-free with `lake build`; `grep -rn sorry` over `formal/Sunfish/` finds nothing outside prose.

Table 1: **Proved theorems** (sorry-free; axioms `propext`, `Quot.sound` only). "—" means unconditional over all games of Definition 2.1; listed hypotheses are named premises carried in the Lean statement (Section 13). *Discharged* means the premise was once assumed and is now derived, so it no longer appears in any shipped-design statement. In-band means $-M_{\mathrm{U}} < \gamma \le M_{\mathrm{U}}$. Supporting lemmas are omitted; `formal/README.md` has the complete list.

Lean theorem	Hypotheses	Informal statement
`Bound.lean` — the core contract (Section 4)		
`bound_spec` (Thm 4.2)	—	The fail-soft null-window search satisfies its docstring against negamax, at every depth, position and window.
`searchMoves_spec`	—	Loop invariant: the running *best* is a fail-soft-correct report of the running true maximum.
`Tricks.lean` — transposition table, futility, mate tricks (Sections 7, 13)		
`boundTT_spec`	Bounded	Table threading preserves TableOK (every stored entry brackets $\mathrm{negamax}_d(p)$) and satisfies BoundSpec.
`futilityOK_discharged` (Thm 7.1)	score identity	The futility yield *equals* the depth-0 child search it replaces.
`boundFut_spec` (Thm 7.2)	FutilityMateOK, in-band γ	The futility-pruned search satisfies BoundSpec against plain negamax — a point spec.
`soften_null_window`	$\gamma > M_{\mathrm{L}} + 1$	Mate-distance softening never changes a fail-high decision.
`mateEntry_deep_service`	MateDepthMonotone	A mate-band lower bound may be served at any *deeper* depth.
`EvalBounds.lean` — numeric facts, by `decide` over the shipped tables (Section 11)		
`evalBound_lt_MATE_LOWER`	—	**Discharges *Bounded***: no static evaluation reaches the mate band.
`kingGone_check_leaked`, `margin_covers`	—	The M_{L} margin leak, and its shipped repair $M_{\mathrm{L}} = K - 13Q$ (Section 11).
`kingCapture_val_above`	—	**Backs *KingCaptureValHigh***: a king capture's move value clears M_{L} even after the worst mover drop.
`quietDropMax_eq`, `capture_terms_nonneg`	—	The move-value floor is -192 (backs *ValFloor*).
`Driver.lean` — what windows MTD-bi can actually probe (Section 4.2)		
`driver_wide_is_now_the_range`	—	**Discharges the in-band window premise**: with the widened bracket, *every* probe window — carried ones included — lies in $(1-M_{\mathrm{U}}, M_{\mathrm{U}}]$, with no clamp and no assumption on scores.
`dstep_wide_sides`	—	Both bracket endpoints stay inside the wide band after every update.
`Stalemate.lean` — the king-capture model, the correction, and the current design (Sections 5–10). *Historical models* `boundStale` (Thm 5.4) and `boundA1` are retained with their specs; the rows below are the *shipped* design.		
`bound_null_spec` (Thm 11.1)	KillerLegal (*a theorem*), driver window	**Layer 1.** The search brackets its own null-inclusive declared value function. *No chess premise.*
`nullValue_eq_realValue_of_noUgramang`	Ugramang	**Layer 2.** The declared function collapses onto the real-move value. This is the only place chess is assumed.

continued on the next page

Lean theorem	Hypotheses	Informal statement
`production_eq_reference` (Thm 9.1)	KingCaptureValHigh, CaptureFirst (*discharged*), KillerLegal	The shipped consumer computes *exactly* the executable reference specification, at every depth and driver-range window.
`kcx_no_crossing` (Thm 11.2)	as above	Two driver-range probes of the same key never store contradictory bounds. Layer-1 only: zugzwang can never cross the table.
`kingCapturableReportsExactAreaboved`	—	The invariant refuted for the old loop (Cex 8.1), restored *by construction* and proven for the shipped consumer.
`kingCaptureContract_stratified`	KingCaptureValHigh, CaptureFirst	The contract in the two strengths the code keeps: fail-high at the QS leaf, exact sentinel above it (Section 9.2).
`futility_iff_child_standpat`	—	$\text{pos.score}+\text{val} < \gamma \iff 1-\gamma \leq -(\text{pos.score}+\text{val})$. One `omega`; the identity the stratification rests on.
`boundary_window_decisive`	—	A fail-soft-sound report at the window $1 - M_{\text{L}}$ classifies mate-band membership *exactly*, both ways — the band-edge arm (Section 10).
`killerLegal_lifecycle` (Thm 6.4)	—	***KillerLegal* is a theorem**: the whole `tp_move` lifecycle — three store species, FIFO eviction, cross-search persistence — preserves the invariant from the empty table.
`captureFirst_of_sorted`	MovesSortedByVal	**Discharges *CaptureFirst*** from the sort specification plus the two value margins.
`legalityProbeCorrect`	KingCaptureValHigh, EvalQuiet	At window M_{U} the dedicated probe is a *complete decision procedure* for "this move left our king capturable".
`storedMoveLegal`	in-band γ	A real fail-high certifies its move legal; legality is position-intrinsic, so the certificate never goes stale.
`scan_sites_unreachable_at_capturable`		Neither correction-scan site ever sees a board with the opponent king capturable.
`Killer.lean` — the killer-table invariant (Section 6)		
`boundKill_spec` (Thm 6.2)	in-band γ, captures sort first	One call preserves KillerOK and returns exactly M_{U} at every king-capturable node.
`killerEmpty_OK`	—	The invariant holds from the empty table along any execution history.
`killer_probe_sound`	—	The depth-0 probe decides king-capturability (both directions) at the depth the engine runs it.

Deleted with their mechanisms (commit `7fdd741`; Sections 7 and 7.5): `LmrDet.lean` — deterministic LMR — proved `boundLmrDet_spec` (a point BoundSpec against the single value function negamaxDet), `boundLmrDet_no_crossing`, and the `clamp_noop_high/low` no-op theorems; `Lmr.lean` — re-search LMR (`58883ea..7f9f164`) — proved `bound_no_crossing` (Thm 7.3) and the crossing counterexample `lmr_tt_crossing` (Cex 7.4). Earlier still, `boundLmr_spec`, `lmrVal_sandwich`, and `IntervalTableOK` with its preservation theorems stated an "interval spec" for re-search LMR and were *deleted as vacuous* (Section 7): no bound on the $V^{\text{hi}} - V^{\text{lo}}$ gap is provable, so they guaranteed nothing; the clamp model (`TableClamp.lean`) went with them once the clamp was proven a no-op under point specs. Git is the archive; the development carries no historical artifacts.

`CanNull.lean` — null move, repetition, keyed table, IID (Section 13.1)		
`boundNullTT_spec`	Bounded	Layer 1, no zugzwang hypothesis: the augmented search brackets its own value nullValue (point spec) and the (pos, depth)-keyed table stays consistent.
`ctableOK_empty`	—	The empty keyed table satisfies the invariant for *any* history (why `tp_score` is cleared on history change).
`rootProbe_spec`	—	The driver and IID probes — unstored in *both* directions — bracket their own value and cannot disturb the table, which is why (pos, depth) remains a complete key with no flag.
`nullValue_plain`	NullBetOK, empty history	Layer 2: nullValue collapses to the null-free value; composed with layer 1, this recovers the docstring.

Table 2: **Machine-checked counterexamples.** Each is a concrete finite game whose search values Lean's kernel computes by `decide`; each is a theorem of the form "the quantified claim is false". They carry the same evidential weight as Table 1 and did the most to correct the authors' beliefs (Section 16). Three of them — the A1 masking bug, the terminal crossing, and the refuted exactness invariant — correspond to *real contradictory transposition entries observed on real boards*; those witnesses are in Table 3.

Lean counterexample	**Refutes**	**Witness**
`lmr_tt_crossing` (`Lmr.lean`, deleted with the mechanism at `7fdd741` — git history; Cex 7.4, Fig. 5)	Point-consistent `tp_score` entries under re-search LMR.	3-position game: the same (pos, depth) yields a fail-high report $+10$ strictly above a fail-low report -50.
`boundStale_not_unconditional` (`Stalemate.lean`; Cex 5.5, Fig. 4)	Soundness of the stalemate correction *without* MateValuesAreKingCaptures (hypotheses (i)–(iii) alone do not suffice).	7-position game, $\gamma = -5$: the corrected search returns an "upper bound" -20 while the draw-aware value is 0.
`negamaxDraw_depth_inconsistent` (`Stalemate.lean`; Thm 5.6)	Existence of a depth-independent game value agreeing with the depth-gated negamaxDraw.	A stalemate position worth *LOSS* at remaining depth 2 and 0 at depth 3.
`cex_violates_hypothesis` (`Stalemate.lean`)	(Companion fact:) that the 7-position game satisfies Hypothesis 5.3.	$\text{negamaxDraw}_2(m_0) = M_{\text{U}}$ with no king capture behind it — a fabricated mate.
`extended_value_not_key_independent` (`Tricks.lean`)	`ExtKeyIndependent`: that the extended-search value is a function of (pos, depth) alone.	2-state game with a history-dependent extension: same key, two values; hence such a TT key must include the history component.
`a1_unfixed_not_sound` (`Stalemate.lean`; Section 8.1)	Soundness of the pre-repair loop, in which a null-move yield could mask the terminal sentinel.	A genuine stalemate at which a *sound* fail-low null yield hides the sentinel; the returned upper bound is exceeded by the true value 0. Every hypothesis, including the null bet, holds. `a1_fix_repairs` shows the repaired loop returns the exact 0.
`cexT_crossing` (`Stalemate.lean`; Section 8)	`TerminalPseudoSafe`: that a pseudo-option may score above a terminal value at a terminal node.	A machine-checked table crossing on the A1-repaired loop — the fail-high dual of the bug above, and what retired that design. `kcx_repairs_cexT` shows the current loop returns the exact 0 at both windows.
`kingCapturableReportsExact_refuted` (`cexR_two_windows`, `Stalemate.lean`; Cex 8.1)	`KingCapturableReportsExact` *for the pre-repair loop*: that a king-capturable node always reports the mate sentinel.	The same move reported as -368 at one window and -69290 at another: under a general fail-soft window a king-capturable child may soundly cut off on stand-pat or a partial table lower without ever mentioning the mate band. *Restored by construction after the repair* (Table 1).
`reducedScan_needs_premise` (`Stalemate.lean`; Section 9)	That the correction's legality scan may skip moves below the quiescence threshold.	A perfectly legal child whose own legal moves all hide below its QS threshold returns exactly $-M_{\text{U}}$ as its filtered value. This refuted a proposed optimization *of ours*, after it had been written; full coverage was restored.
`stratLeaf_needs_futility` (`Stalemate.lean`; Section 9.2)	That the depth-0 contract may be weakened without modeling futility as a distinct yield species.	A depth-1 terminal node whose child's stand-pat cuts: the real-move fold lands above the sentinel, the correction cannot fire, and the node returns a value strictly below the exact draw. The shipped engine is unaffected — which is *why* futility had to become a modeled yield species rather than a documentary caution.

continued on the next page

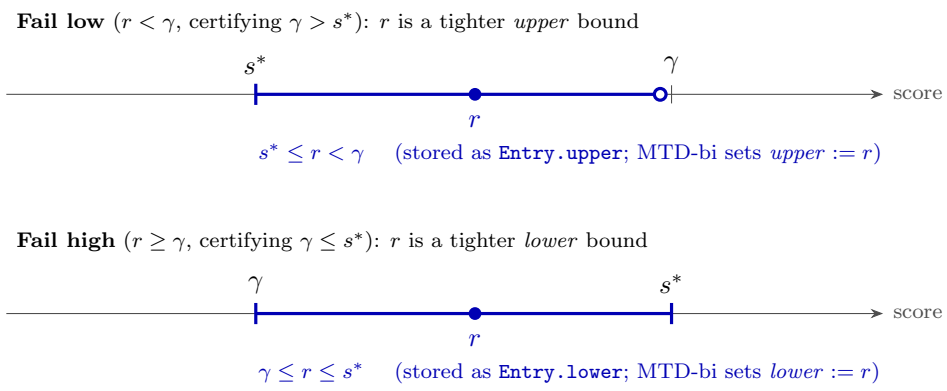

Figure 2: BoundSpec (Definition 4.1) as a picture: wherever the report r falls relative to the probe window γ, it is trapped on the correct side of the true value $s^* = \text{negamax}_d(p)$. Fail-soft means r carries information beyond the boolean fail-high/fail-low: it is itself a valid bound, which is what the transposition table stores and what MTD-bi's bracket consumes (Figure 3).

Lean counterexample	Refutes	Witness
`carried_gamma_escapes_band` (`Driver.lean`; Section 4.2)	The README's and the code comment's own claim that MTD-bi only probes $(-M_{\text{L}}, M_{\text{L}}]$.	After a mate-band score, the first probe of the next depth carries a γ above the band. Fixed by widening the bracket, not by weakening the theorems.

4 The Core Contract

4.1 The specification, transcribed verbatim

The docstring of `Searcher.bound` (sunfish.py, lines 286–290) reads:

```
Let s* be the "true" score of the sub-tree we are searching.
The method returns r, where
if gamma >  s* then s* <= r < gamma  (A better upper bound)
if gamma <= s* then gamma <= r <= s* (A better lower bound)
```

In the model of Section 2, the "true score" s^* is the depth-limited negamax value $\text{negamax}_d(p)$, and the docstring transcribes to a two-sided fail-soft condition on the report r, illustrated in Figure 2. Here $M_{\text{U}} = 69290$ and $M_{\text{L}} = 50710$ are sunfish's `MATE_UPPER` and `MATE_LOWER`.

Definition 4.1 (BoundSpec, the docstring)**.** For a game G, depth d, position p, window γ and report r,

$$\text{BoundSpec}(G, d, p, \gamma, r) \;:\equiv\; \big(\gamma \le r \to r \le \text{negamax}_d(p)\big) \;\wedge\; \big(r < \gamma \to \text{negamax}_d(p) \le r\big).$$

(`BoundSpec`, `Bound.lean`)

The search itself, stripped to its logical skeleton, is the fail-soft loop with the early cutoff on *best* $\ge \gamma$, over children searched with the flipped null window $1 - \gamma$ one ply shallower (`bound`, `searchMoves`, `Bound.lean`). No table, no null move, no quiescence, no killer: those return in Sections 5–13.

Theorem 4.2 (The docstring is true)**.** *For every game G, depth d, position p and window $\gamma \in \mathbb{Z}$,* $\mathrm{BoundSpec}\big(G, d, p, \gamma, \mathrm{bound}_d(p, \gamma)\big)$ *holds. (`bound_spec`, `Bound.lean`)*

Proof sketch. Induction on depth. The loop is handled by a single invariant lemma (`searchMoves_spec`, `Bound.lean`): *the running best is itself a fail-soft-correct report of the running true maximum.* On a cutoff, *best* $\geq \gamma$ came either from the current child (whose report is $\leq$ its true value by the induction hypothesis) or from the previous *best* (bounded by the invariant); either way the fold over the unsearched tail can only grow. Without a cutoff, both the child and the previous *best* failed low, so both upper-bound their true counterparts and the invariant is re-established. The null-window step is the integer fact $b < 1 - \gamma \iff \gamma \leq -b$: a child failing low against $1 - \gamma$ fails high for the parent, and vice versa. The proof is sorry-free; `#print axioms` reports only `propext` and `Quot.sound`. □

4.2 Why this is the load-bearing spec: the MTD-bi driver

Sunfish's driver `Searcher.search` is MTD-bi (a bisecting variant of MTD(f) [8]): at each depth it maintains a bracket $\mathit{lower} \leq \mathit{score} \leq \mathit{upper}$, initialized to $[1 - M_{\mathrm{U}}, M_{\mathrm{U}}]$, and repeatedly probes `bound` at $\gamma = \lfloor(\mathit{lower} + \mathit{upper} + 1)/2\rfloor$, stopping when the bracket is narrower than `EVAL_ROUGHNESS` ($= 15$ centipawns) — Figure 3. Theorem 4.2 is exactly the property the binary search needs: *every* probe, whatever its γ, moves one end of the bracket — a fail high raises *lower* to r, a fail low lowers *upper* to r, and the two can never cross (Theorem 7.3 below makes the no-crossing claim formal, and Section 7 shows how the since-removed re-search Late Move Reductions broke precisely it). Fail-softness is what makes bisection converge fast in practice: the report r typically overshoots γ, so the bracket shrinks by more than half.

Two details of the driver are load-bearing for later sections. The first is that the bracket invariant is exactly what the LMR crossing counterexample (Cex. 7.4) broke — and, because the spread of the crossing reports admits no provable bound, there was no slack (not `EVAL_ROUGHNESS`, not re-probing) that could honestly be said to absorb it (Section 7) — the fact that eventually killed the mechanism (Section 7.5).

The second is the *window range*, and it is a small cautionary tale about believing a comment. Almost every theorem downstream — the stalemate correction, the killer invariant, the current layered specs — carries a hypothesis of the form "γ is in band". That hypothesis is discharged only if the driver never probes outside the band, and for a long time both this paper's source and the engine's own comment asserted that MTD-bi only probes $(-M_{\mathrm{L}}, M_{\mathrm{L}}]$. Modeling the driver (`Driver.lean`) showed the assertion is *true for every window computed at the current depth while scores stay strictly inside the band* (`driver_band_invariant`) and *false* for the first probe of a depth after a mate-band score: sunfish carries γ across the per-depth bracket reset, and a forced-mate score carries it above the band (`carried_gamma_escapes_band`, machine-checked).

There were two ways to close the gap: clamp γ, or widen the bracket to an interval that already contains everything the driver can produce. The engine shipped the clamp first, then replaced it: the bracket is now the full flip-closed band $[1 - M_{\mathrm{U}}, M_{\mathrm{U}}]$ and the clamp is deleted. The payoff is unconditional — `driver_wide_is_now_the_range` and `dstep_wide_sides` prove that *every* probe window, carried ones included, lies in $(1 - M_{\mathrm{U}}, M_{\mathrm{U}}]$, with no clamp and no assumption about scores — so the in-band hypothesis is discharged once, at the driver, for every theorem that consumes it. Widening also fixed a latent bug the model surfaced in passing: a fail-soft mate score exceeds any narrower upper end, which inverts the bracket. The cost is nil, because fail-softness means the first probe collapses the bracket wherever it started.

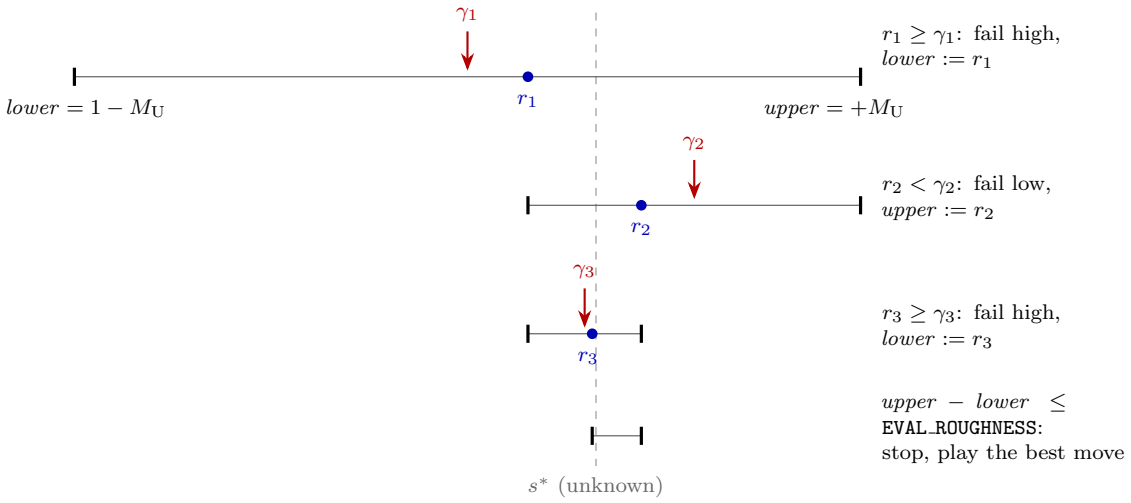

Figure 3: MTD-bi as a binary search over the probe window γ. Each row is one call $r_i =$ `bound`(pos, γ_i, d) with $\gamma_i = \lfloor (lower + upper + 1)/2 \rfloor$. Theorem 4.2 guarantees each report lands on the correct side of the (unknown) true value s^*, so every probe legitimately moves one end of the bracket, and fail-soft overshoot (e.g. $r_1 > \gamma_1$) shrinks it faster than plain bisection. The invariant $lower \leq s^* \leq upper$ is Theorem 7.3; Cex. 7.4 showed what the since-removed re-search LMR did to it.

5 The King-Capture Model and the Stalemate Correction

Sunfish generates *pseudo-legal* moves and scores an actual king capture as an immediate win: a position whose static score is $\leq -M_L$ has already lost its king and is normalized to exactly $-M_U$. "No legal moves" (mate *or* stalemate) therefore surfaces in the search as $best = -M_U$ after the loop, and the engine post-processes it (at depth > 2) with an in-check probe on the null-moved position: $best := -M_L$ if in check, else 0.

Definition 5.1 (Draw-aware value)**.** negamaxDraw$_d(p)$ is negamax over the king-capture-normalized game with the depth-gated stalemate correction applied to the fold: if $d > 2$ and the fold equals $LOSS$, the value becomes $-M_L$ if p is in check and 0 otherwise. (`negamaxDraw`, `drawFix`, `Stalemate.lean`)

Definition 5.2 (CheckProbeOK)**.** A probe function agrees pointwise with the one-ply king-capture reading of check: after passing the turn, the opponent has a move reaching a position with eval $\leq -M_L$. For the real engine this is the assumption that the depth-0 QS probe `bound(flipped, MATE_UPPER, 0) == MATE_UPPER` decides check — true because quiescence never prunes king captures. (`CheckProbeOK`, `inCheckB`, `Stalemate.lean`)

Named Hypothesis 5.3 (MateValuesAreKingCaptures)**.** Whenever negamaxDraw$_d(p) = M_U$ with $d \geq 1$, some move of p actually reaches a position with eval $\leq -M_L$: mate-band sentinel values always come from a real king capture, never from a shallow-horizon artifact. (`MateValuesAreKingCaptures`, `Stalemate.lean`)

Theorem 5.4 (The corrected search satisfies the docstring)**.** *Let G be a game with a pass move, and suppose: (i) all evaluations lie in $[-M_U, M_U]$ (*Bounded*); (ii) the probe satisfies*

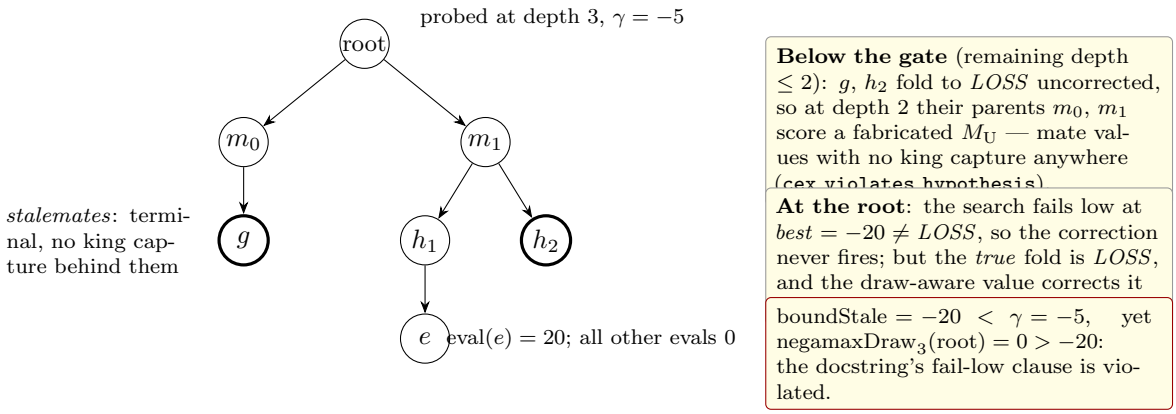

Figure 4: The seven-position counterexample `Cex` (Cex. 5.5). Thick circles are terminal stalemates; the depth > 2 gate means they are scored *LOSS* when first seen at shallow remaining depth, their parents inherit fabricated mate scores, and the root reports an upper bound the draw-aware value exceeds. Bounded evaluations, a perfect check probe and an in-band window all hold; only Hypothesis 5.3 fails — so the hypothesis is not optional.

CheckProbeOK*; (iii) the window is in band:* $-M_{\mathrm{U}} < \gamma \leq M_{\mathrm{U}}$ *(an interval closed under the null-window flip* $\gamma \mapsto 1 - \gamma$*); and (iv)* MateValuesAreKingCaptures*. Then the stalemate-corrected search* boundStale *satisfies* BoundSpec *against* negamaxDraw *at every depth and position. (`boundStale_spec`, `Stalemate.lean`)*

Proof sketch. The same loop invariant as Theorem 4.2, plus a pointwise lemma (`staleFix_spec_core`, `Stalemate.lean`) showing the correction maps a fail-soft-correct pair (search *best*, true fold) to a fail-soft-correct pair. The crux is the masking step: if the *true* fold is *LOSS*, then (by the band, the sentinel invariant, and Hypothesis 5.3) every child is an exact king-capture mate, every child report is exactly *LOSS*, and hence the search's *best* is *LOSS* too — so both sides apply the *same* correction, using the correct probe. In-band windows are needed so that a fail-low at *LOSS* certifies the loop ran to completion: a $\gamma \leq -M_{\mathrm{U}}$ would allow an early cutoff at *LOSS* with unsearched moves remaining. Sorry-free. □

Each hypothesis is individually necessary; the interesting one is (iv):

Counterexample 5.5 (The correction is not unconditionally sound)**.** There is a seven-position game with bounded honest evaluations, a perfectly correct probe, and an in-band window $\gamma = -5$, on which the faithful depth-gated correction returns a fail-low "upper bound" of -20 while the draw-aware value is 0. The mechanism: two positions are terminal stalemates; seen at remaining depth ≤ 2 (gate off) they score *LOSS*, so their parents at depth 2 score a fabricated M_{U} — mate scores with no king capture anywhere behind them — and the root inherits a wrong bound. This is sunfish's own code comment ("sunfish may report 'mate', but then after more search realize it's not a mate after all") turned into a machine-checked refutation. (`boundStale_not_unconditional, Cex, cex_violates_hypothesis`, `Stalemate.lean`)

Theorem 5.6 (Depth inconsistency is intrinsic)**.** *The* depth > 2 *gate makes the draw-aware value itself depth-dependent: in the counterexample game there is a stalemate position g with* negamaxDraw$_2(g) =$ *LOSS and* negamaxDraw$_3(g) = 0$*. No depth-independent game value can agree with* negamaxDraw *at every depth. (`negamaxDraw_depth_inconsistent`, `Stalemate.lean`)*

Remark 5.7 (Why mate is $-M_{\mathrm{L}}$, not $-M_{\mathrm{U}}$). $-M_{\mathrm{U}}$ is the reserved sentinel meaning "the king is *actually* capturable/gone". Scoring checkmate-by-correction as $-M_{\mathrm{U}}$ would re-inject the sentinel at positions whose king is not capturable and destroy the parent-level detection one ply up; it would also order "mated next move" above "king already lost". The engine's choice of $-M_{\mathrm{L}}$ in the correction is what keeps the sentinel trustworthy, and the distinction is load-bearing for everything in Sections 8–11: $-M_{\mathrm{U}}$ is a *claim about the board*, $-M_{\mathrm{L}}$ is a claim about the game. The campaign's central bug is what happens when the two are conflated.

Remark 5.8 (The margin was almost too small). Modeling the numbers rather than the algorithm produced one shipped repair. For the sentinel argument to work, the mate band must be wide enough that no static evaluation can reach it; the engine originally set $M_{\mathrm{L}} = K - 10Q$. Checking that by `decide` over the transcribed piece-square tables found a leak (`kingGone_check_leaked`): a position with the king gone *and* enough queens could evaluate above $-M_{\mathrm{L}}$, so the "have we still got a king?" test would miss. The repair, $M_{\mathrm{L}} = K - 13Q$ (now 47,923), is proven to close it (`margin_covers`), and the general fact — no static evaluation reaches the band — is `evalBound_lt_MATE_LOWER`, which is what discharges *Bounded* for the shipped tables rather than assuming it.

6 The Killer Invariant: Conjectured at Noon, Proved by Evening

While modeling the stalemate correction we recorded a potential gap: the killer move (the stored `tp_move` entry) is yielded *before* the sorted moves, so a non-capture killer failing high could let `bound` return a value below M_{U} at a king-capturable position — breaking the sentinel requirement that Theorem 5.4 leans on. The maintainer conjectured this cannot happen in sunfish. The conjecture was proved the same day.

Definition 6.1 (KillerOK, the table invariant)**.** A killer table t (a partial map from positions to moves) satisfies KillerOK if every stored entry is a legal move of its key position, and at a king-capturable position the stored entry — if any — is *itself a king capture*. (`KillerOK`, `Killer.lean`)

Theorem 6.2 (KillerIsKingCapture)**.** *Thread* `tp_move` *through the search as state (position-keyed, store-on-fail-high-only), with the move loop running over value-ordered moves (king captures, value $\geq M_{\mathrm{L}}$, strictly first). Then for every depth d, position p, in-band window $-M_{\mathrm{U}} < \gamma \leq M_{\mathrm{U}}$, and table t satisfying* KillerOK*:*

1. *the call* $\mathrm{boundKill}_d(p, \gamma, t)$ *returns a table satisfying* KillerOK*; and*
2. *if $d \geq 1$, p's own king is not already gone, and p is king-capturable, the returned value is* exactly M_{U} *— on the killer path and on the loop path alike.*

(`boundKill_spec`, `Killer.lean`)

Proof sketch. Induction on depth, cases on the table lookup. *Killer-cutoff case*: the stored move is the killer itself; by the invariant it is a king capture whenever p is capturable, the king-capture normalization scores it at the full sentinel, and an in-band γ makes the sentinel fail high — so the cutoff both stores a king capture and reports exactly M_{U}. *Killer-fail-low case*: a king-capture killer would have reported $M_{\mathrm{U}} \geq \gamma$, so p is not capturable and plain loop preservation applies. *No-killer case*: at a capturable position the capture heads the ordered list, cuts off immediately at the

sentinel, and nothing else can be stored first (`killLoop_capture_head`, `Killer.lean`). Applied along any execution history starting from the empty table (`killerEmpty_OK`, `Killer.lean`), the invariant holds forever. Sorry-free; axioms `propext`, `Quot.sound` only. □

Corollary 6.3 (Exact sentinel; fail-low certifies non-capturability)**.** *At any king-capturable node (depth ≥ 1, in-band window) the search reports exactly M_{U}; contrapositively, any in-band report $< M_{\mathrm{U}}$ — in particular any fail low — certifies that no king capture was available. This is what makes the depth-0 stalemate probe of Section 5 meaningful.*

Three conditions are load-bearing, and each is an edit a well-meaning PR could make:

- **Position-exact keying.** Every store into `tp_move`$[p]$ happens inside a call at p itself; a ply-shared killer table (a common engine idiom) would let a quiet killer harvested at one position cut off at a different, king-capturable one, breaking the induction.
- **Captures sort first.** King captures have value $\geq M_{\mathrm{L}}$, strictly above every quiet move.
- **In-band windows.** An out-of-band $\gamma > M_{\mathrm{U}}$ could let a quiet move cut off (and be stored) above the capture's sentinel.

The residual exception is the null-move yield: it stores nothing (so KillerOK survives) but can end the loop below the sentinel; sunfish guards it only by $|\text{pos.score}| < 500$, a guard its own comment concedes is heuristic. The condition under which the guard closes the hole is the named hypothesis (`NullGuardBlocksAtCaptures`, `Killer.lean`). Closing it *properly* — by validating the yield instead of guarding it — is what Section 9 does.

Empirical companion. Before the proof existed, the conjecture was checked empirically on the production engine: **0 violations in 694,533 killer cutoffs over 1,270 real-game positions**. The proof upgraded "empirically absent" to "impossible given in-band windows and value ordering" — and, usefully, told us exactly which future edits would void the guarantee.

6.1 From invariant to lifecycle

Theorem 6.2 is a statement about *one call* from an invariant-satisfying table. That is the right shape for an induction but the wrong shape for a guarantee about a running engine, which stores from several places, evicts under memory pressure, and — because `tp_move` is deliberately *not* cleared between searches — carries entries across move boundaries. Every one of those is an opportunity to store something the invariant forbids.

The current development therefore proves the stronger statement directly, over the store trace:

Theorem 6.4 (The killer table's whole lifecycle)**.** *Let* KillerInv *be the position-intrinsic invariant "every stored entry is a legal move of its key, and at a king-capturable key it is itself a king capture". Then for* any *finite sequence of store events drawn from the engine's three store species — the fail-high store of a searched real winner, the substitution store of a verified king capture, and the mate-case futility store — interleaved with FIFO evictions, folding the sequence from the empty table yields a table satisfying* KillerInv*; hence* KillerLegal *holds of the resulting lookup function. (`killerLegal_lifecycle`, `Stalemate.lean`)*

Proof sketch. Induction over the event list; the base case is the empty table, and each step is discharged by the species' own certificate. A fail-high store is justified by `storedMoveLegal`: a real report at or above an in-band γ can only have come from actually searching that move, so the move is legal — and legality is a property of the *position*, not of the search that discovered it, which is precisely why the entry cannot go stale when reused at a later depth or in a later search. The substitution store writes a move that the consumer has just verified to be a king capture (Section 9). The mate-case futility store writes a move whose value reaches the mate band, and `HighValIsKingCapture` turns that into a king capture. Eviction only deletes, and a position-intrinsic invariant is preserved by deletion. Nothing in KillerInv mentions search state — no depth, no window, no node counter — which is exactly why the invariant survives across searches, and exactly why exact-position keying is load-bearing. □

The practical yield is a rule with teeth: **a pull request that adds a store site must name its event species and supply that species' certificate**, and one that ply-shares the table — a common engine idiom — loses the induction outright. The upgrade also changes the status of *KillerLegal* everywhere downstream: it is no longer a premise of the shipped design's specs but a theorem they may cite (Table 1).

7 A Soundness Taxonomy for Selective Search

The campaign's conceptual yield is a classification of how a selective search may consult its window γ. Sunfish's maintainer stated it as a principle, which we quote verbatim:

> "γ may shape termination, and may trigger shortcuts whose value provably one-side-bounds the same function; it must never select between incomparable evaluations."

The three classes, each with its canonical citizen:

7.1 Class (a): γ-shaped termination — always sound

The beta cutoff itself: γ decides only *when the loop stops*, and fail-soft reporting keeps every report a bound on the same negamax. This is Theorem 4.2; nothing further is needed. Null-window search, iterative deepening, and aspiration re-probing all live here.

7.2 Class (b): γ-triggered shortcuts that one-side-bound the same function — sound

Futility pruning is the canonical citizen. At depth ≤ 1, once $\text{pos.score} + \text{val}(m) < \gamma$, sunfish answers with the *static estimate* $\text{pos.score} + \text{val}(m)$ instead of searching the child. The estimate is below γ, so it is only ever a fail-low report, and BoundSpec demands of a fail-low report exactly one inequality: $-\text{negamax}_d(m) \leq \text{pos.score} + \text{val}(m)$.

We initially stated that inequality as a hypothesis (*FutilityOK*). It was then *discharged*: sunfish builds child positions incrementally, and the score identity

$$\text{eval}(m) \;=\; -\big(\text{eval}(p) + \text{val}(p, m)\big) \qquad (m \in \text{moves}(p))$$

is literal in `Position.move`. With the identity as a structural property of the model (`ValGame`, `Tricks.lean`), the futility test $\text{pos.score} + \text{val} < \gamma$ is integer-equivalent to "the child's stand-pat

meets its window $1-\gamma$", at depth 0 the stand-pat is yielded first and fails high immediately, and fail-soft returns exactly the child's score:

Theorem 7.1 (FutilityOK discharged — as an equality)**.** *In any game with the score identity, for every position p and move $m \in \mathrm{moves}(p)$,*

$$-\,\mathrm{negamax}_0(m) \;=\; \mathrm{eval}(p) + \mathrm{val}(p, m).$$

The futility yield is not an estimate: it equals *the depth-0 child search it replaces. (`futilityOK_discharged`, `Tricks.lean`)*

Theorem 7.2 (Futility-pruned search satisfies BoundSpec)**.** *For any game with the score identity, under in-band windows and the remaining hypothesis* FutilityMateOK *(a move valued as a king capture,* $\mathrm{val} \geq M_{\mathrm{L}}$*, really wins: the fail-high `else MATE_UPPER` bypass), the futility-pruned search satisfies* BoundSpec *against plain* negamax *— a point spec on a single value function. (`boundFut_spec`, `Tricks.lean`)*

Proof sketch. At depth 1 (the futility zone) each per-move yield is analyzed by cases: a futile quiet move yields exactly $-\mathrm{eval}(m)$ by Theorem 7.1; a futile king capture yields the exact sentinel, a fail high justified by *FutilityMateOK* and the in-band window; a non-futile move is an exact depth-0 child. A member-restricted variant of the loop lemma (`searchMoves_spec_mem`, `Tricks.lean`) (the score identity only speaks about legal moves) closes the loop; deeper depths are Theorem 4.2's argument verbatim. Sorry-free. □

Fine print made explicit by the model: the *quantified* FutilityOK (over all depths) is not dischargeable — plain negamax at $d \geq 1$ has no stand-pat option — but the search only ever consumes the $d = 0$ instance, and that instance is the score identity on the nose. The original statement over-required.

7.3 Class (c): γ-selected incomparable evaluations — unsound in the point sense

Re-search Late Move Reductions — the gamma-adaptive variant that shipped between commits `58883ea` and `7f9f164`, and whose formal record was kept in `Lmr.lean` until mechanism and model were deleted together at `7fdd741` (Section 7.5; git history) — is the canonical citizen. A late quiet move at depth ≥ 3 is probed at depth -2; a reduced fail low is yielded as-is, a reduced fail high is re-searched at full depth and only that result is yielded. Here γ *selects which of two incomparable value functions the yield is a fact about*, and the docstring becomes unprovable as stated: no single value function backs both sides.

Our first reaction was to retreat to an interval-shaped statement. Define the mutually recursive pair $(V^{\mathrm{hi}}, V^{\mathrm{lo}})$ with side alternation:

$$V_d^{\mathrm{hi}}(p) = \max_m \; -V_{d-1}^{\mathrm{lo}}(m) \qquad \text{(all full depth)}$$

$$V_d^{\mathrm{lo}}(p) = \max_m \begin{cases} \min\bigl(-V_{d-2}^{\mathrm{hi}}(m),\, -V_{d-1}^{\mathrm{hi}}(m)\bigr) & \text{if } m \text{ reducible} \\ -V_{d-1}^{\mathrm{hi}}(m) & \text{otherwise} \end{cases}$$

over an abstract reducibility predicate $\mathrm{red}(d, i, m)$ generalizing sunfish's `depth >= 3 and i_m >= 5 and val < QS`. We proved, machine-checked and sorry-free at the time (as `boundLmr_spec`

and `lmrVal_sandwich`), that every fail-high report of the LMR search is $\leq V_d^{\mathrm{hi}}(p)$, every fail-low report is $\geq V_d^{\mathrm{lo}}(p)$, and $V_d^{\mathrm{lo}}(p) \leq \mathrm{negamax}_d(p) \leq V_d^{\mathrm{hi}}(p)$ pointwise — and, for a while, we presented this pair of statements as "the honest replacement" for the docstring: an *interval spec*, with the truth trapped in between.

The verdict must be stated as plainly as the retreat was: **it was never a spec**, and the statements — together with the $(V^{\mathrm{hi}}, V^{\mathrm{lo}})$ definition itself — have been *deleted* from the development. A claim of the form "fail highs bound V^{hi}, fail lows bound V^{lo}, and the truth lies in between" is a guarantee only if the gap $V^{\mathrm{hi}} - V^{\mathrm{lo}}$ is bounded. No bound on search instability is provable: one ply of depth can hide a mate, so the gap is unbounded precisely in the positions that decide games. A guarantee relative to an uncontrollable gap guarantees nothing; the interval reading was only a description of the bug — in the maintainer's phrase, a "cope spec". The only contract the repository recognizes is the *point* spec: every stored bound describes one value function determined by the transposition key. What `Lmr.lean` kept, until the mechanism itself was removed and the file deleted with it (Section 7.5), was exactly what supported that doctrine: the re-search definition itself (`boundLmr`) and the machine-checked counterexample below.

Two facts about modeling the since-deleted statements were *surprises* — both were claims the authors believed and the proof assistant refused; we recount them honestly in Section 16.

The practical consequence is a transposition-table pathology:

Theorem 7.3 (No crossing without LMR)**.** *For the unreduced search, a fail-high report can never exceed a fail-low report at the same* $(\mathrm{pos}, \mathrm{depth})$*: both bracket* $\mathrm{negamax}_d(p)$*, so a* `tp_score` *entry always satisfies lower* $\leq$ *upper. (*`bound_no_crossing`*, proved in* `Lmr.lean` *and deleted with the file at* `7fdd741`*: for the now-shipped full-width search the fact is immediate from Theorem 4.2 — both reports bracket the same* $\mathrm{negamax}_d(p)$*.)*

Counterexample 7.4 (TT crossing under LMR, machine-checked)**.** There is a three-position game and windows $\gamma = -60, 0$ on which the LMR search at the same $(\mathrm{pos}, \mathrm{depth})$ produces a fail-high report of $+10$ and a fail-low report of -50: a `tp_score` entry with $\mathit{lower} = 10 > -50 = \mathit{upper}$. The single root move looks bad shallow but is good deep; probing $\gamma = 0$ takes the reduced branch and fails low at -50, probing $\gamma = -60$ bypasses it, re-searches at full depth and fails high at $+10$. (`lmr_tt_crossing`, `Lmr.lean`; deleted with the mechanism at `7fdd741` — in git history at `58883ea..7fdd741`.)

MTD-bi's bracket $\mathit{lower} \leq \mathit{score} \leq \mathit{upper}$ is therefore simply broken under re-search LMR: the gap between the two value functions admits no provable bound, so there is no theorem of the form "the crossing stays within what `EVAL_ROUGHNESS` and re-probing absorb" — in the positions that decide games, it need not. On the production engine the crossing is real but rare: **0.016% of nodes** measured over real games. The two-line store clamp shipped at the time — on store, widen the stale side of the entry instead of asserting a contradiction — measured ELO-neutral at $+12 \pm 28$; but it was containment, not repair: it kept crossing reports from being *stored* (proved at the time as `clamp_no_crossing`) while the search went on consuming them. The `IntervalTableOK` invariant that once dressed the clamp up as preserving a guarantee was deleted along with the interval reading — an invariant stated relative to an unboundable gap certifies nothing — and the clamp model (`TableClamp.lean`) has since been deleted from the development entirely; git is the archive. Killer and stalemate lemmas were unaffected while the mechanism shipped: king captures have val $\geq M_{\mathrm{L}} > QS = 40$ and were never reducible (`RedRespectsCaptures`, deleted with `Lmr.lean`); the killer is pre-loop and structurally never reduced; reduced fail lows are $< \gamma$ and never reach the `tp_move` store; and the $-M_{\mathrm{U}}$ king-loss

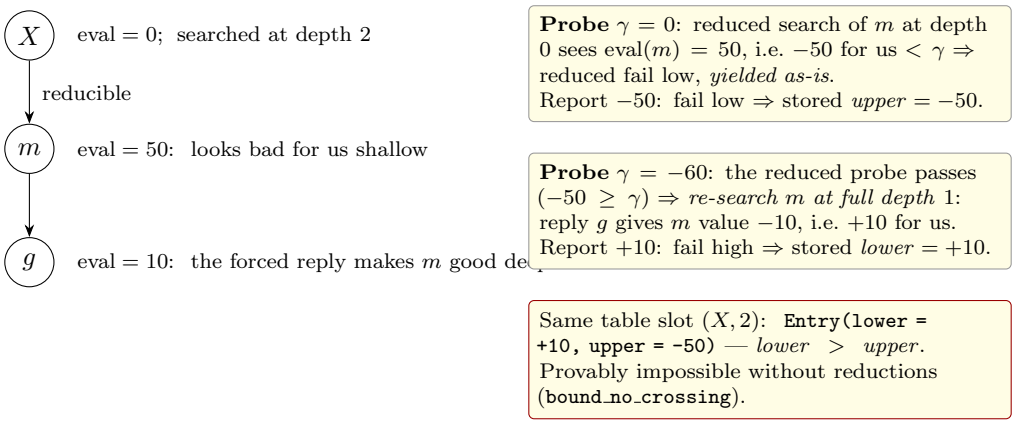

Figure 5: The machine-checked TT-crossing counterexample `lmr_tt_crossing` (Cex. 7.4). The window γ decides whether the yield for move m is a fact about the *reduced* (depth-0) or the *full* (depth-1) evaluation — two incomparable value functions — so two probes of the same $(\text{pos}, \text{depth})$ return bounds that cross when stored. Mechanism and Lean artifact are both gone from the working tree (Section 7.5); the counterexample survives in git history — and here, as the explanation of why re-search LMR died.

sentinel is depth-independent, so stalemate detection still saw exact sentinels from reduced searches.

7.4 First epilogue: deterministic LMR restores the point spec

The story seemed, for a day, to have its resolution. At commit `7f9f164`, master retired the re-search variant and shipped *deterministic* LMR: the reduction depended only on (depth, index, val) — `LMR = int(depth >= 4 and i_m >= 8 and val < 0)` — and each move was searched *once*, at depth $-1-$ LMR. Because γ no longer selected between evaluations, the search was back in class (b′): it computed exact fail-soft bounds on a *single* value function negamaxDet (reducible moves valued one ply shallower, recursively), with a point spec (`boundLmrDet_spec`, `LmrDet.lean` — proven at the time, deleted with the mechanism, Section 7.5); fail-high reports could never exceed fail-low reports at the same key, so transposition entries were contradiction-free by construction (`boundLmrDet_no_crossing`); and the store clamp of Cex. 7.4's containment became a provable no-op (`clamp_noop_high/low`, proven at the time) and was removed from the engine — after which the no-op theorems and the whole clamp model (`TableClamp.lean`) were themselves deleted from the development, under the maintainer's principle that git is the archive and the code carries no historical artifacts. The price was believed measured, not guessed: the gamma-adaptive re-search variant appeared about 16 ELO stronger (-16 ± 38 direct; both large wins over no LMR, or so the ledger then read — Table 5). That trade — 16 ELO for end-to-end point specs on single value functions — was made deliberately. Note that negamaxDet $\neq$ negamax: deterministic LMR was honest about *which* value function it computed bounds on, rather than unsound about the original one. It was also, as the next subsection shows, mispriced — in both directions at once.

7.5 Second epilogue: the measurement campaign removes the mechanism entirely

The resolution of Section 7.4 did not survive its next measurement campaign (all matches per `docs/TESTING.md`: frozen snapshots, paired openings, per-engine time-loss accounting). Three results, in the order they dismantled the trade:

- **Removing deterministic LMR is a gain, not a price.** No-LMR beat the shipped deterministic LMR by $+69 \pm 40$ and $+29 \pm 33$ (two 300-game matches at 4+0.04) and by $+19 \pm 45$ at 60+1 (200 games) — stronger at every time control tested, with fixed-depth suites corroborating (mate-in-4, Bratko–Kopec and Win-at-Chess all improve without LMR). Deterministic LMR's believed -16 ± 38 price against re-search was really ~ -50: net negative against *no* LMR at all. The methodological lesson went into `docs/TESTING.md`: a decision that hinges on a difference smaller than its error bars needs SPRT or a larger sample.
- **The historical ledger was never wrong.** Replaying the exact A/B that merged re-search LMR at `58883ea` ($+49 \pm 34$ then) reproduces today: $+41 \pm 36$.
- **The edge was the bug.** Re-search LMR grafted onto today's baseline still measures $+20 \pm 36$ over no-LMR — but an *honest* variant with identical cutoffs and sound min-semantics bound propagation (each stored bound a claim about the minimum over the depths actually searched, key-determined by construction) scores exactly 0.00 ± 34 against no-LMR. The entire measured value of re-search LMR lived in its over-claimed bounds: fail highs propagated as facts about a depth they did not search. That is not a trade-off between strength and provability; it is strength borrowed from a bug — the very transposition-table contradiction machine-checked in Cex. 7.4 while the mechanism existed.

Commit `7fdd741` therefore removes Late Move Reductions entirely. The move loop is again a single full-width `for val, move in sorted(...)`, every move searched at depth -1; with no reduction anywhere, `bound`'s docstring is provable exactly as written — Theorem 4.2, the point spec of `Bound.lean`, now describes the shipped loop with no caveats — and the formal record needs no LMR machinery: `Lmr.lean`, `LmrDet.lean` and `TableClamp.lean` were deleted with the mechanism, under the same git-is-the-archive principle that had already claimed the interval spec. What survives in the development is what the episode taught, stated once in `formal/README.md`: every stored bound must describe one value function determined by the transposition key; "the truth lies within an unboundable gap" is not a specification.

8 A Score Is Not a Legality Proof

The first half of this campaign found bugs in a *mechanism* — reductions — and removed it. The second half found a bug in a *habit of reasoning*, present in three different places, and removing it required changing how the search justifies its terminal decisions. This section states the habit and exhibits the three refutations; Section 9 is the repair.

Sunfish detects mate and stalemate the way most king-capture engines do. Because legality is enforced one ply late, "no legal move" is not directly observable; instead the move loop folds per-move scores into a running *best*, initialized to the fold's identity element $LOSS = -M_{\mathrm{U}}$, and after the loop the engine reasons:

best is still LOSS, so no move improved on "our king is captured"; so every generated move loses the king; so there are no legal moves; so this node is mate or stalemate and I may overwrite the fold with an exact terminal value.

Every step of that chain is a *score* being read as a statement about *legality*. It is sound only if two things hold: every yield that can raise the fold comes from actually searching a real move, and a searched real move reports above the sentinel exactly when it is legal. In an engine with selective search, neither holds. The move loop also emits *virtual* options — the null move, the quiescence stand-pat, futility estimates — which are evidence about *value* and carry no information whatsoever about whether the side to move has a legal move. A virtual option that outscores the sentinel silently converts "this node is terminal" into "this node is not terminal".

8.1 A1: a sound null-move yield masks a real stalemate

The first instance is a fail-*low* masking. At a genuine stalemate, the null-move option is searched and returns some value above $-M_{\mathrm{U}}$ — soundly, since passing really is available to the model of the position the null move creates. That yield raises *best* off the sentinel. The correction's guard ($best = -M_{\mathrm{U}}$) is therefore false, the correction does not fire, and the node returns a fail-low report as an upper bound on a value that is really 0. The report is wrong in the direction that matters: it is an *upper* bound below the truth.

This is machine-checked as `a1_unfixed_not_sound`, and the statement is deliberately unflattering to the old design: the counterexample satisfies *every* hypothesis then on the books, including the null-move bet. The bug is not zugzwang. It is a category error in the fold.

The repair modeled at the time separated the accumulators: a `best_real` that only searched real moves may raise, tested by the correction, alongside the null-inclusive *best* that determines the return value (`boundA1`, with `a1_fix_repairs` confirming the exact 0). It shipped, and it was not enough — because the dual exists.

8.2 The fail-high dual, and three assumptions refuted on real boards

If a virtual option can hide a terminal node by scoring *above* the sentinel, it can also hide one by scoring above the terminal *value*. The correction is an override: it *assigns* $best := 0$ (or $-M_{\mathrm{L}}$ if in check) after the loop. A pseudo-option that scored, say, +11 at a position whose true value is 0 does not merely mask the correction — it produces a fail-high report of +11 at a node whose exact value is 0, and a fail-low probe of the same node stores 0. That is a crossed transposition entry: $lower = 11 > 0 = upper$.

We isolated this as a named obligation, `TerminalPseudoSafe`, and machine-checked that it does not come for free (`cexT_crossing`). Its instances then turned out to be not merely unproven but *false on real boards*. Each was hunted with a search script over real positions rather than argued about; each produced a witness and a genuine contradictory entry from the shipped engine of the day.

Counterexample 8.1 (Exactness is not free)**.** For the pre-repair loop, `KingCapturableReportsExact` is false: there is a game and two windows at which the same king-capturable node reports two different values, neither of them M_{U}. (`kingCapturableReportsExact_refuted, cexR_two_windows`, `Stalemate.lean`)

The lesson is worth stating in the form the repository now keeps it, because it is the design rule the rest of the paper follows:

Assumption	What it claimed	Refutation (real board, real entry)
`NullAtStalemate-Nonpositive`	At a genuine stalemate the null-move pass cannot score positive, so it cannot outbid the exact draw.	`8/8/8/5k1p/3b1P1P/1p1P1P1P/pN1P1P2/K7 w`, static score +175: the pass honestly consumed +11 at a genuine stalemate. The engine stored `Entry(lower=11, upper=0)`. Mechanism: at null-depth 1 a re-frozen stalemate evaluates as *stand-pat*, not as 0.
`StandPatAtTerminal`	At a terminal node the quiescence stand-pat cannot be the fold's winner.	`6Rk/6QP/8/8/8/8/8/K7 b` — every pseudo-move a defended capture. Stored `Entry(lower=-1711, upper=-47923)`. A dedicated search script found 100+ further natural corner mates with the same shape.
`KingCapturable-ReportsExact`	A node at which the opponent king can be captured always *reports* exactly M_{U} — so a report below M_{U} proves no capture was available.	False under general fail-soft windows: such a node may soundly cut off on its stand-pat, or on a partial table lower bound, without ever mentioning the mate band. Traced as the same move scoring -368 at one probe and -69290 at another; machine-checked as `cexR_two_windows`.

Table 3: Three "score implies legality" assumptions, each refuted on a real board with a real crossed transposition entry. The third is the most consequential: it is the invariant on which the previous two sections' arguments were quietly resting, and its failure is the root cause of the whole family. All three are retained in the development as countermodels that nothing consumes.

> **A fail-soft score is evidence about value, not evidence about legality. Mobility is certified only by a legal fail-high move or by a dedicated legality probe. When a bound would cross a terminal value without such a certificate, verify before storing it.**

9 Verify-on-Suspicion

The repair is not a patch to the correction's guard; it is a change to what the move loop *records*. Three ideas, in the order they matter.

1. Yields have species, and the species is load-bearing. The engine's move generator emits five kinds of yield, and they divide into two groups. *Real*: a searched move's result, and the mate-case futility yield (which is a genuine king capture handed out at the sentinel rather than pruned). *Virtual*: the null move, the quiescence stand-pat, and sub-mate futility estimates. Real yields are evidence about both value and legality; virtual yields are evidence about value only. In the shipped code a virtual yield is literally tagged — it is emitted with `None` in the move slot — and in the model the distinction is structural (`futileAt`, `searchedAt`, `futTerm` keep synthetic suffix estimates out of the real accumulator).

This was documentation before it was a mechanism, and the gap between those two states cost a real bug: sub-mate futility estimates used to be emitted as real yields, which produced the crossed entry `Entry(lower=0, upper=-1054)`. Making them virtual fixed it.

2. The fold carries a certificate. Alongside the value accumulator the loop maintains one boolean, `live`: *some real move has been proven legal.* A searched real move's score is two-way evidence — exactly $-M_{\mathrm{U}}$ proves the move left our own king capturable, anything above proves it legal — so the certificate costs one comparison per yield and no searching at all. In the model this is the test $S = LOSS$ on the real-only accumulator, and `searched_yield_two_way` is the lemma that the two readings agree.

3. Verify only where a bound would otherwise be unjustified. Two places can produce an unjustified bound, and each gets a check sized to what it needs.

The consumer interception. Before a *virtual* yield is allowed to end the loop, it is validated. If a real king capture exists, it is *substituted*: the node reports M_{U} through an ordinary real cutoff and `tp_move` stores the true capture — active preservation of the killer invariant, not merely non-destruction. A mate-band claim with no capture behind it is vacuous (if passing wins the king, capturing it is a real move too) and is folded to the identity. A positive claim at a node already verified terminal would outscore the exact draw and is likewise folded away. The mate-band and terminal arms are depth-gated and the substitution arm is not, and that asymmetry is not cosmetic: folding a terminal stand-pat at depth 0 makes a quiescence node *return* the reserved sentinel, which we measured as 96 mismatches before the gate was placed correctly.

The correction. If the node failed low and no real move was proven legal, it is *suspect* — either it is terminal, or a virtual option outbid every legal move — and only then does the engine pay for a direct proof: probe every generated move and ask whether the reply can capture our king.

Two details of that scan matter more than they look. It runs over the *full* generated list, not the quiescence-filtered one. We proposed the filtered version as an optimization, and it is unsound: a perfectly legal child whose own legal moves all hide below its quiescence threshold returns exactly $-M_{\mathrm{U}}$ as its filtered value, so reading scores instead of re-deriving legality reintroduces the very inference this section exists to remove. That is `reducedScan_needs_premise`, and it refuted our own proposal after it had been written. And the check is a *board predicate* — "does this reply capture a king?" — not a search probe. The resulting doctrine is a one-liner with real consequences for the code: **board predicates for legality; search probes only where a search value is genuinely needed.** Exactly one search probe survives in the terminal logic, and Section 10 explains why it must.

9.1 A reference specification, and a proof that the shipped loop equals it

The design above is easier to specify than to implement. The *specification* wants an eager entry scan: at any king-capturable node, return M_{U} immediately, before the table, before the repetition check, before anything. The *implementation* cannot afford that — a full legality scan at every node is not a price a 147-line Python engine can pay — so it earns the same guarantee lazily, through the interception, the move order, and the killer table.

Rather than argue that the two agree, the development contains both and proves it. `boundD2` is the reference (and a runnable `reference.py` accompanies it in the repository); `boundKCX` is the shipped loop.

Theorem 9.1 (Production equals reference)**.** *Under `KingCaptureValHigh`, `CaptureFirst` and `KillerLegal` — the latter two themselves theorems (Sections 6.1, 11) — the shipped consumer*

and the reference compute the same function *at every depth, position, and driver-range window.* *(`production_eq_reference`, `Stalemate.lean`)*

Proof sketch. Strong induction: children one level down, the pass three levels down. At a king-capturable node the reference returns M_{U} by construction; the production loop reaches the same value by one of two routes, and the case split is exactly the two routes — either a virtual yield arrives first and the interception substitutes the capture, or no virtual yield cuts and the capture heads the sorted move list (`CaptureFirst`) and cuts as the first searched real yield. Everywhere else the interception and the reference's verifier make identical decisions about the shared pass probe (`nullArm_match`), and the two folds agree child by child. □

The theorem is the machine twin of an engine-side check we ran anyway: reference and production compared bound-for-bound over 9,600 probes. Neither replaces the other — the battery covers the Python the model abstracts, the theorem covers the infinitely many positions the battery does not — but agreeing is what makes both believable.

Its immediate corollary is the invariant that started Section 8: `kingCapturableReportsExact_restored` re-establishes, now as a theorem about the shipped consumer, the statement Cex. 8.1 refuted for the old loop. The countermodel stays in the development as the record of why the repair was necessary.

9.2 Stratifying the contract, and one arithmetic identity

One piece of the repair is worth isolating, because it is where a proof directly bought back code.

The interception originally had a depth-0 arm, so that a quiescence node could also promise the exact sentinel. That arm is unnecessary, and the reason is a single line of integer arithmetic. The contract is *stratified*: at depth 0 a king-capturable node need only *fail high*, and at depth ≥ 1 it reports the sentinel *exactly* (`kingCaptureContract_stratified`).

The depth-0 half is free: either the stand-pat already meets the window, or the capture does with room to spare (`qsStrat_failHigh_of_capture`). The question is whether the weaker leaf costs the layers above anything, and the answer is no, because of:

Theorem 9.2 (The futility test *is* the child's stand-pat test)**.** *For all integers,*

$$\mathrm{pos.score} + \mathrm{val} < \gamma \quad \Longleftrightarrow \quad 1 - \gamma \;\leq\; -(\mathrm{pos.score} + \mathrm{val}).$$

(`futility_iff_child_standpat`, `Stalemate.lean`)

Proof. `omega`. □

The identity is trivial; its consequence is not. Read through the score identity of `Position.move`, it says that a parent *searches* a child exactly when that child's own stand-pat cannot cut. So the only depth-0 children the recursion ever reaches are those at which the quiescence loop must run past its stand-pat — into the king capture, which heads the move order — and therefore reports the sentinel exactly anyway (`qsStrat_exact_of_searched`). The weaker leaf is invisible to every consumer, and the interception's depth-0 arm could be deleted.

The same identity, incidentally, is the sentence sunfish's own source has carried for a decade as a comment justifying futility pruning: `"pos.score + val < gamma === -(pos.score + val) >= 1-gamma"`. It was a theorem all along.

Remark 9.3 (What the split cost, and what it required). This is also the clearest example in the campaign of the model having to *improve* rather than merely record. With the leaf weakened, the model's encoding of the certificate — "no searched real yield beat the sentinel" collapsed to "the accumulator is still *LOSS*" — was no longer faithful, and `stratLeaf_needs_futility` machine-checks the resulting hole. The shipped engine was never affected (futility fires on exactly the offending child, so the yield is virtual and the certificate stays false), but the model had to make futility a first-class yield species before the stratified contract could be stated honestly. Under the repository's rule that the model must always do what the code does, that was not optional.

10 The Last Chess Premise, and Why We Could Not Assume It

After the repair, one chess statement remained inside the correctness layer. Call it `NoZugzwangInMateBand`:

> *If passing wins in the mate band, then some real move does too — you cannot be in zugzwang while delivering forced mate.*

It is the mate-band fragment of the ordinary no-zugzwang assumption, and it is needed because the interception's mate-band arm tests the pass's *report* for band membership, while reports straddle the band across different windows: a sub-band report can be a loose bound on a mate-band value.

It is also extremely plausible, which is why it survived several review passes. Delivering forced mate while having no move at all that does so sounds self-contradictory.

It is false. A counterexample search over generated positions, verified against `python-chess`, produced `8/6p1/6R1/k7/2K5/8/8/8 w`: legal, Black has exactly one reply, passing mates immediately, and White has no forced mate within three. Three sibling positions with the same property were found alongside it.

That is a different situation from every other named hypothesis in this paper. A bet you cannot discharge can be named, measured and kept; a premise that is *false* and *load-bearing* cannot be kept at all. The choice was to remove the need for it.

The band-edge probe. When a sub-band virtual cutoff would otherwise survive, the engine re-probes the pass at one specific window — the band boundary $1 - M_{\mathrm{L}}$ — and uses the direction of that report to decide. The window is chosen so that the report is decisive in *both* directions:

Theorem 10.1 (The boundary window is decisive)**.** *Let r be any fail-soft-sound report for value V at the window $1 - M_{\mathrm{L}}$, i.e. $1 - M_{\mathrm{L}} \le r \Rightarrow r \le V$ and $r < 1 - M_{\mathrm{L}} \Rightarrow V \le r$. Then*

$$r < 1 - M_{\mathrm{L}} \quad \Longleftrightarrow \quad M_{\mathrm{L}} \le -V.$$

(`boundary_window_decisive`, `Stalemate.lean`)

Proof. Both directions are integer arithmetic on the two fail-soft implications. Left to right: a fail low gives $V \le r < 1 - M_{\mathrm{L}}$, hence $M_{\mathrm{L}} \le -V$ (with $M_{\mathrm{L}} \ge 0$, `omega` closes it). Right to left: if r were $\ge 1 - M_{\mathrm{L}}$ then $r \le V \le -M_{\mathrm{L}}$, which contradicts $M_{\mathrm{L}} \ge 1$. In Lean the whole proof is a `constructor` and two `omega` calls. □

The point is that at a general window a report only bounds the value on one side, so mate-band membership is undecidable from it; at *this* window the two sides meet, and the single report classifies membership exactly. The engine therefore *certifies* what the premise asserted, at the cost of one extra probe on a path that is already rare.

The result. With the band-edge arm shipped, `NoZugzwangInMateBand` leaves the premise list. Layer 1 — the search's bracketing property against its own declared value function, and with it the non-crossing of the transposition table — **carries no chess statement at all**. Zugzwang survives only in layer 2, as the validity region of the null-move approximation, where it governs the *accuracy* of the value and can never make the table contradict itself. The false premise is retained in the development as a record, with its witness, because "we assumed this and it was false" is exactly the kind of thing a formal development should be unable to forget.

11 The Invariant Catalogue: What Is Guaranteed, and How

This section is the paper's answer to the question a maintainer actually asks: *what does the shipped engine guarantee, what earns each guarantee, and how do they compose?* It is self-contained.

11.1 The invariants

I1. The search brackets its declared value. For every depth, position and driver-range window, the report r satisfies $\gamma \le r \Rightarrow r \le V(p,d)$ and $r < \gamma \Rightarrow V(p,d) \le r$, where V is the engine's null-inclusive declared value function — one function determined by the transposition key, gamma-free and killer-free. *Guaranteed by:* `bound_null_spec` (Theorem 11.1), transferred to the shipped consumer by `production_eq_reference`. *Chess premises:* none.

I2. The table cannot contradict itself. Two probes of the same $(\text{pos}, \text{depth})$ at any two driver-range windows never produce a fail-high report above a fail-low report; hence no stored entry has *lower* > *upper*. *Guaranteed by:* `kcx_no_crossing` (Theorem 11.2) — an immediate corollary of I1, which is the entire point of insisting on a point spec. *Chess premises:* none.

I3. King capture is reported faithfully. At depth ≥ 1, a node whose opponent king can be captured returns exactly M_{U}; at depth 0 it fails high. *Guaranteed by:* `kingCaptureContract_stratified`, assembled from `qsStrat_failHigh_of_capture` (leaf) and `boundKCX_capture_exact` (interior), with `kingCapturableReportsExact_restored` stating it for the shipped consumer. *Earned by construction* through the substitution arm — this is the invariant that was refuted before the repair (Cex. 8.1).

I4. The killer table only ever holds proven moves. Every `tp_move` entry is a legal move of its key position, and at a king-capturable key it is itself a king capture — across all store species, eviction, and successive searches. *Guaranteed by:* `killerLegal_lifecycle` (Theorem 6.4). *Status:* was a premise, is now a theorem.

I5. Static evaluations never reach the mate band. No position's static score can be confused with a king-capture sentinel. *Guaranteed by:* `evalBound_lt_MATE_LOWER`, decided by the kernel over the transcribed piece-square tables, together with the $M_{\mathrm{L}} = K - 13Q$ margin repair (`margin_covers`, Remark 5.8).

I6. Every probe window is in band. Every window the driver can ever pass to `bound` — including the one carried across a depth boundary — lies in $(1 - M_{\mathrm{U}}, M_{\mathrm{U}}]$. *Guaranteed by:* `driver_wide_is_now_the_range` and `dstep_wide_sides`, with no clamp and no assumption about scores (Section 4.2).

11.2 How they compose

Theorem 11.1 (Layer 1: the algorithm, unconditionally)**.** *For every game, guard and killer predicate satisfying* KillerLegal*, and every depth, position and window in* $(-M_{\mathrm{U}}, M_{\mathrm{U}}]$*, the reference search brackets* nullValueD2 *in the fail-soft sense. (`bound_null_spec`, `Stalemate.lean`)*

Proof sketch. Strong induction on depth — strong, because the pass option recurses at depth -3 rather than depth -1. Depth 0 and the two by-construction branches (king already gone; king capturable) are reflexivity, since search and declared value agree definitionally there. The interesting case is a verified null cutoff, and it splits on whether the oracle has confirmed the node terminal. If it has, the surviving cutoff is non-positive (`positiveNullCutoffVerified` — the positive case was folded away by the interception) and the in-check sub-case is impossible outright, because an enabled pass at an in-check node is exactly the fold identity (`nullAtMateD2`). If it has not, the declared function admits the pass term as its initial accumulator and the cutoff is bounded by it — either by the suppression on a fail high, or by the window on a fail low. The remaining case is the ordinary fold, which is Theorem 4.2's argument with two additions: futile children contribute through the virtual accumulator, and their declared contribution is covered by `futile_contrib_le`, which is why the declared function needs no gamma and stays key-determined. □

Theorem 11.2 (No crossing)**.** *Under the same premises, if a probe at* γ_1 *fails high and a probe at* γ_2 *fails low, then* $\mathrm{bound}(p, \gamma_1, d) \le \mathrm{bound}(p, \gamma_2, d)$*. (`kcx_no_crossing`, `Stalemate.lean`)*

Proof. The fail-high report is $\le V(p, d)$ and the fail-low report is $\ge V(p, d)$ by Theorem 11.1; `omega`. The content is entirely in there being *one* V — which is what the point spec buys and what re-search LMR could not provide (Section 7). □

Theorem 11.3 (Layer 2: accuracy, and the only chess assumption)**.** *Under* NoZugzwang — *"the pass value never exceeds the best real move" — the declared value function coincides with the real-move value, and the search's guarantee becomes a guarantee about chess. (`nullValue_eq_realValue_of_noZugzwan`, `boundKCX_spec`, `Stalemate.lean`)*

Figure 6 shows the dependency structure. The shape is the point: everything a *table* can depend on lives in the chess-free layer, so a failure of the chess assumption can make the engine play a worse move but cannot make its data structures inconsistent.

11.3 What this does *not* say

Three limits, stated plainly, because a catalogue like this is easy to over-read.

First, the guarantees are about the *Lean model*. The bridge to the Python is audit plus the automated drift guard of Section 12, not machine proof.

Second, the scoped abstractions of Section 2.4 remain: notably quiescence's interior is the leaf's fixpoint rather than being unrolled, and `Position`'s internals are axiomatized by structural

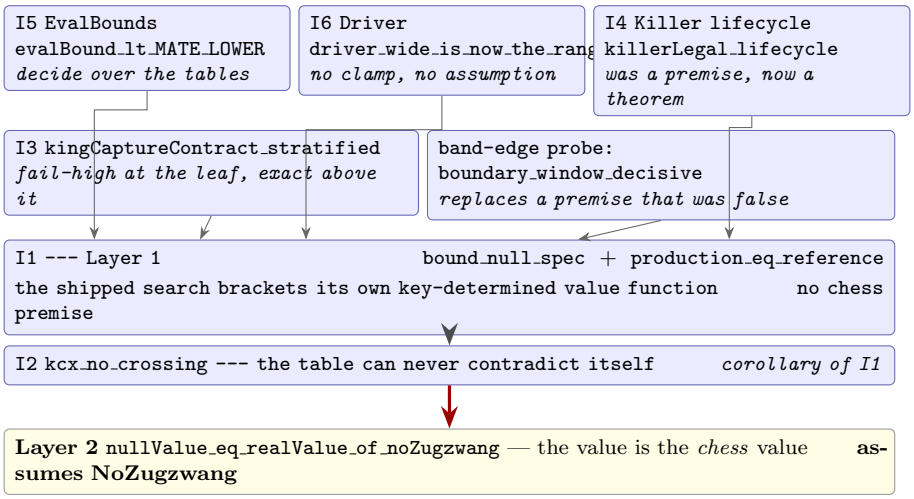

Figure 6: The correctness chain. Everything above the thick rule is chess-free: the numeric facts, the driver's window range, the killer lifecycle, the king-capture contract and the band-edge probe compose into I1, and I2 falls out as a corollary. Only the final step — identifying the engine's declared value with the real chess value — assumes anything about chess, and no table entry depends on it.

properties. "`gen_moves` implements chess" is not proved here and is the one genuinely open item (Section 16).

Third, `HistoryLegal` — that positions in the game history are never king-capturable — is accepted as *input validity* rather than proved: it closes the one path that precedes the consumer, since a king-capturable position inside the repetition set would answer with a draw score and evade I3.

The honest one-sentence summary, which is also how the pull request describes itself: *the search's bound property and its table's consistency are machine-proven about the shipped code with no chess assumptions, over a model that treats quiescence's interior and `Position`'s internals as axiomatized boundaries.* That is a much stronger claim than the project has made before, and it is not "the whole engine is verified".

12 Keeping the Model Honest

A formalization of a live codebase decays unless something prevents it. The rule this repository adopted — *the model must always do what the code does*, with divergence treated as a blocker rather than a footnote — needs enforcement, and two failure modes turned out to need different mechanisms.

Semantic drift is caught by hashing. `formal/scripts/model_audit.py`, run in CI, hashes every audited region of `sunfish.py` — `Searcher.bound`, `Searcher.search`, the `Position` methods the model axiomatizes, and the constants — and fails on any change that is not accompanied by a same-commit re-audit and hash refresh.

Citation drift is invisible to hashing, and we only noticed because it had already happened: a Lean comment citing "sunfish.py:339" is not code drift when line 339 moves, so no hash changes, and `Killer.lean` had rotted exactly that way — citing 339 and 356–357 for code that had moved

to 391 and 422–423. The guard now also (a) checks that every distinctive source *anchor* the model cites by name still occurs in `sunfish.py`, and (b) *ratchets* the number of raw line-number citations across the Lean sources so it can fall but never rise. New comments are expected to cite function names and distinctive source text instead of line numbers.

Neither mechanism proves the correspondence. They make an *undetected* divergence require someone to actively defeat CI, which is a different risk from the one we started with.

13 The Ledger of Bets: Named Hypotheses

Every pruning trick that is only conditionally sound is kept out of the proven core and given a *named hypothesis*: the condition, not the induction, is the interesting content. The discipline this buys is a repository rule: **a search-changing PR must identify which lemma it preserves, weakens, or newly depends on** (see `formal/README.md` for the full PR-guideline ledger). Every unproven assumption is named, stated, and where possible measured.

The ledger's most useful property is that it *moves*. Table 4 is the current state of the premises the shipped search rests on; the status column is the interesting one, because nearly every row has changed status at least once during this campaign, and three rows changed status by being *refuted*.

Three entries deserve a sentence each. `ExtKeyIndependent` is stated as the *false* claim a (pos, depth)-keyed table silently makes once search extensions depend on history (e.g. a recapture extension keyed on the last-capture square); a two-state counterexample refutes it, and the consequence is that such a TT key must include the history component. Sunfish now avoids the problem structurally: its quiescence re-derives capture information from the position alone, and the one deviant semantics that remains — the driver and IID probes' no-null, no-repetition view — never touches the table in *either* direction, so (pos, depth) is a complete key with no flag at all. (It was not always: a `can_null` component sat in the key for years, and was removed once its only consumers stopped needing the table. The general test a new flag must pass is in Section 17's pointer to the PR guideline: either the flag is *provably table-invisible*, or it selects a genuinely second value function and must join the key and pay for the partitioned hit rate.) Finally `soften_null_window` (`soften_null_window`, `Tricks.lean`) is a small proven lemma: for windows strictly above the mate band, mate-distance softening does not disturb null-window fail-high decisions.

13.1 Shrinking a bet: the layered null-move story

The ledger is not static; the null-move entry shows a bet being *shrunk* by restructuring (Figure 7). Sunfish's `can_null` flag plays four roles at once — null-move gate, repetition gate, transposition-key component, and IID recursion flag — and `CanNull.lean` models all four exactly as master runs them (including two audit surprises recorded there: sunfish permits *consecutive* null moves, and the move generator's laziness is semantically load-bearing for the table state). The old, monolithic claim "null-move search satisfies the docstring" needed a zugzwang hypothesis in front of everything. The layered version needs it almost nowhere:

- **Layer 1 (unconditional).** Define nullValue — the value function of the null-and-repetition-augmented search itself, with the pass option (searched at depth -3 behind the gamma-free guard $|\text{score}| < 500$) as one more option in the fold. Then the search brackets nullValue with a *point* spec, and the (pos, depth, can_null)-keyed table stays consistent,

Premise	Statement (informal)	Status
Layer 1 — the algorithm and the table		
`Bounded`	Static evaluations lie in the score band.	**Discharged** for the shipped tables (`evalBound_lt_MATE_LOWER`)
`KillerLegal`	A `tp_move` entry away from a king-capturable node is a legal move.	**Theorem** (`killerLegal_lifecycle`, Thm 6.4)
in-band window	$\gamma \in (1 - M_{\mathrm{U}}, M_{\mathrm{U}}]$ at every probe.	**Discharged** at the driver (`driver_wide_is_now_the_range`)
`CheckProbeQuiet`	The in-check probe decides check.	**Discharged**; the test is now the board predicate `rotate().king_capture()`
~~`NoZugzwangInMateBand`~~	You cannot be in zugzwang while delivering forced mate.	**Discharged by the band-edge arm** (Sec. 10) — and **false in chess**, witness `8/6p1/6R1/k7/2K5/8/8/8 w`
Layer 2 — accuracy only; no table entry may depend on these		
`NoZugzwang`	The pass value never exceeds the best real move.	**Named**, stated once, attached to the accuracy lemma
`NullBetOK`	Recursive form of the same bet (`CanNull.lean`).	**Named**; confined to the bridge `nullValue_plain`
Transfer to the shipped consumer		
`KingCaptureValHigh`	King captures are valued $\geq M_{\mathrm{L}}$.	**Backed** by `EvalBounds.kingCapture_val_above`
`CaptureFirst`	King captures head the sorted move list.	**Discharged** (`captureFirst_of_sorted`) from the sort spec
Fidelity / structural — true of the code by construction, pinned by the drift guard		
`ScoreIdentity`	`pos.move` builds the child with the negated summed score.	Structural; what makes Thm 9.2 a statement about stand-pats
`MovesSortedByVal`	Python's `sorted` sorts, descending by value.	**The one trusted primitive**
`HighValIsKingCapture`	A mate-band move value *is* a king capture.	**Backed** by the `EvalBounds` margins
`EvalQuiet`, `ValFloor`	Quiet evaluations stay below the band; move values are ≥ -192.	**Backed** by `EvalBounds`
Input validity		
`HistoryLegal`	Positions in the game history are never king-capturable.	**Accepted** (every reached position came from a legal move)
Refuted — retained as countermodels that nothing consumes		
`TerminalPseudoSafe`	A pseudo-option may not outscore a terminal value.	**Refuted** (`cexT_crossing`) and its two instances refuted on real boards (Table 3)
`KingCapturable-ReportsExact`	A capturable node always reports M_{U}.	**Refuted** for the old loop (Cex. 8.1); **restored as a theorem** after the repair
`NoMaskedMobility`	The correction's scan may skip filtered moves.	**Refuted** (`reducedScan_needs_premise`); our own proposed optimization
`ExtKeyIndependent`	Extended-search value is a function of (pos, depth) alone.	**Refuted** (`Tricks.lean`); hence such a key must include the history component

Table 4: The premise inventory as it stands. "Discharged" means the premise was once carried in a statement and is now derived, so it appears in no shipped-design theorem. The layering is the load-bearing structure: only the two layer-2 rows say anything about chess, and Theorem 11.2 — the table's consistency — does not depend on them.

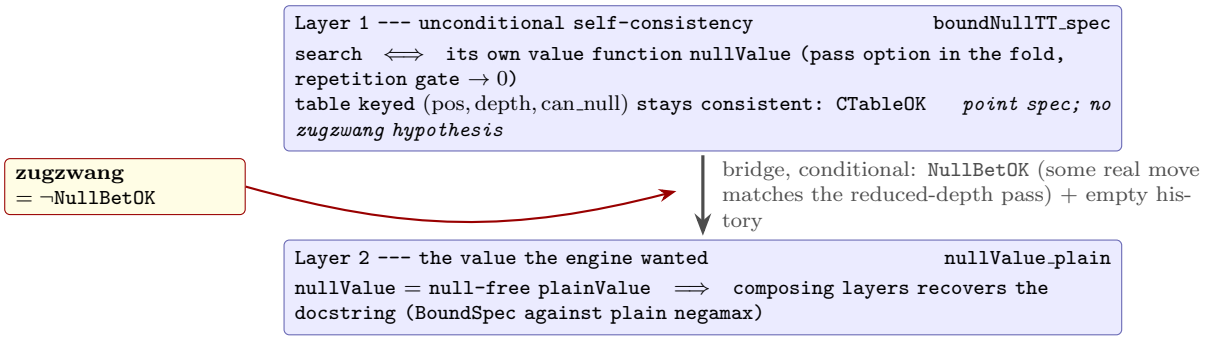

Figure 7: The layered null-move argument (Section 13.1). The unconditional layer proves the search and its keyed table can never disagree with *each other*; the named hypothesis `NullBetOK` is confined to the bridge that identifies the computed value with the null-free one. Zugzwang attacks only the bridge — it can make the value wrong, never the table inconsistent.

with no zugzwang hypothesis anywhere (`boundNullTT_spec, CTableOK`, `CanNull.lean`). Self-consistency of search plus table is not a bet.

- **Layer 2 (the bet, now smaller).** Relating nullValue to the null-free value is where the null-move bet lives: under `NullBetOK` (some real move matches the reduced-depth pass; the required witness also excludes guard-passing stalemates) and an empty history, nullValue collapses to the plain value (`nullValue_plain`, `CanNull.lean`), and composing the layers recovers the docstring.

Zugzwang — precisely ¬`NullBetOK` — can make the engine compute the wrong *value*, but can never make its table contradict its search. A satisfying corollary of the keyed-table invariant being history-relative: the empty table satisfies it for *any* history (`ctableOK_empty`, `CanNull.lean`), which is exactly why sunfish clears `tp_score` whenever the game history changes.

The framing, in one paragraph: the engine is *allowed* to bet — that is what selective search is — but every bet must have a name, a statement in the ledger, and, where possible, a measured exposure. Restructuring can shrink a bet (this subsection); a counterexample can prove a bet is real (Cex. 5.5); and a discharge can retire one (Theorem 7.1). What is not allowed is an unnamed bet.

14 Empirical Methodology and Honest Numbers

Functional tests tell you the engine is correct; they tell you nothing about whether a change makes it stronger. Every strength-relevant change in this campaign was priced by a tournament between engines frozen with `git archive` (code *and* tools — no shared files), playing color-swapped pairs from a random-order opening book under fastchess [2], typically 300 games at `tc=4+0.04` ($\approx \pm 30$ ELO at 95%), with adjudication, on an otherwise idle machine. The full methodology, each rule with its origin story, is `docs/TESTING.md` in the repository.

14.1 Results

What the correctness campaign cost. The headline is that it cost nothing measurable. The verify-on-suspicion landing — which added a legality certificate to the fold, a three-armed

validation before any virtual cutoff, a full-coverage board-predicate scan, and later a band-edge probe — measures -10.4 ± 28.7 over 300 games at 60+1 against the master it replaced: indistinguishable from zero, and the point estimate is well inside the noise floor for that sample. The subsequent golf, deleting the interception's depth-0 arm once Theorem 9.2 showed it redundant, measures $+5.8 \pm 29.0$. Neither number is interesting on its own; together they say the repairs of this campaign were bought with proofs rather than with playing strength, which is not what the LMR story would have led us to expect.

The last row is different in kind: it is the standing price of the one assumption the engine still makes. Deleting the null move outright — which is what refusing the zugzwang bet would mean — costs -34.9 ± 40.3. The engine keeps the bet, the ledger keeps its name, and the formal work's contribution is that the bet is now confined to a layer where being wrong costs *accuracy* and not *consistency*.

Three narrative points, because the numbers alone mislead:

The price of point-consistency. Re-search LMR admits no point spec (Cex. 7.4), and the interval-shaped statement we first proved in its place guarantees nothing (Section 7). One can buy the point spec back — and each way of doing so was priced: deterministic LMR (-48 ± 33), a conservative reduction condition (-16 ± 38), sticky TT entries (-33), ordering-only (-34). Every restoration appeared ELO-negative. Likewise, completing null-move soundness (searching the pass at a depth for which the spec discharges) was priced at -28 and declined: the engine keeps the bet, and the ledger keeps its name (`NullBetOK`). The two epilogues are instructive. Knowing both the price *and* what it buys, the maintainer first chose to pay the smallest one — conservative deterministic LMR, believed $\sim$16 ELO for point specs end-to-end (Section 7.4). Then the final campaign (Section 7.5) showed there had been nothing to pay for: the believed -16 was really ~ -50, and the honest form of what the point spec supposedly cost is worth exactly 0.00. Master now ships no reductions at all — the point spec holds for the code as written, at a measured *gain*. Neither decision — first paying, then unwinding — would have been coherent without both loops: the proofs identified exactly what the ELO was buying (unsound bound propagation), and the tournaments certified that buying it honestly earns nothing.

Time-regime artifacts. An earlier survey measured reverse futility at +45 and mate-score softening at +19. After a fix to time handling (in-search deadlines replacing between-iteration checks), both were re-measured on the fixed baseline: -1 and -22. The original numbers were artifacts of the old time regime, not chess. Rule: after any time-handling change, re-baseline everything you believed.

The multi-TC lesson. An opening think-time ramp (cap of 0.1s$\times$ ply) shipped with no cutoff and silently governed *entire games* at long time controls: at 15+3 the proper budget is $\sim$25s/move but the ramp still bound at move 60, and a user's game ended in mate with 14 of 15 minutes unused. The bug was invisible at the 4+0.04 test TC, where the ramp stops binding at ply 2 — it sailed through 300-game matches. It was caught by a user report and fixed the same day; the fix was verified by driving the per-move time curve directly at 15+3 (17–24s observed mid-game vs. 3–9s before). This became a standing rule: time-management changes must be validated at multiple time controls, including a direct per-move time-curve check, because fast-TC matches are structurally blind to long-TC time bugs.

Contamination stories, briefly. A contaminated run (engines sharing a live checkout) once measured −60 ELO at LOS 0.02% for a change later shown to search bit-identical node counts — hence the freeze rule and the old-vs-old control match. Tournament managers reuse engine processes across games, so state surviving `ucinewgame` acts as invisible cross-game learning: a four-way eviction-policy round-robin ranked +66 / +47 / +15 / −132 across candidates whose *per-game* search was in some cases provably identical. Behavior-neutrality is checked by depth-limited lockstep (moves *and* node counts), never by clock.

14.2 The frame

Proofs identify the trade; tournaments price it. Neither loop alone would have handled LMR correctly: without the proof we would not have known the crossing existed, that no interval reading rescues it, or that the killer and stalemate arguments survive reduction; without the tournaments, the point spec would first have been bought blind — and we would never have learned that its true price was negative: the honest variant of the reductions is worth exactly zero (Section 7.5), so the engine now ships full-width and provable, stronger than it was with either LMR.

15 Related Work

Nipkow's invited ITP 2024 paper [7] and the accompanying Isabelle/HOL AFP entry [6] verify fail-hard and fail-soft alpha-beta over linear orders, distributive lattices and de Morgan domains; the fail-soft correctness statement there is the two-sided bound that our BoundSpec specializes to a null window, and our loop invariant ("the running best is a fail-soft report of the fold so far") follows the same shape. *Formal Verification of Minimax Algorithms* [5] verifies minimax variants with alpha-beta and transposition tables in Dafny — the closest prior treatment of the table invariant (our `TableOK`). Knuth and Moore's classical analysis [4] and the MTD(f) framework [8] supply the algorithmic background; futility pruning in the form sunfish uses traces to Heinz [3].

Our position: to our knowledge this is the first Lean 4 formalization of null-window/MTD-bi search, and the first line of work to pair machine proofs of search soundness with ELO-measured prices on a production engine — one that plays rated games in public. We found no prior formal treatment of king-capture (pseudo-legal) search, of a stalemate correction, of killer-table invariants, or of a machine-checked refutation of transposition-table point-consistency under gamma-adaptive Late Move Reductions (Cex. 7.4).

The verified prior work above shares a convenient property that sunfish does not have: it specifies searches over *legal* move generators, where "no moves" is directly observable and terminal values need no reconstruction. Pseudo-legal, king-capture search — the idiom that makes a 147-line engine possible at all — moves terminality from an observation to an inference, and Section 8 is an account of that inference going wrong in three ways. We are not aware of prior formal work on it, and we suspect the "score implies legality" family is common in engines that use the idiom, precisely because each instance looks locally obvious.

16 Lessons and Threats to Validity

The model–code gap. All theorems are about the Lean model of Section 2, not about `sunfish.py`. The gap is managed, not closed: the model is line-mapped (Figure 1; the cor-

respondence table in `formal/README.md` pairs every Lean definition with the sunfish code it transcribes), it is re-audited at every commit touching an audited region, and CI enforces both halves of that discipline mechanically (Section 12). The counterexamples are constructed to exercise exactly the modeled mechanism, and features excluded from a given model are excluded *explicitly*, from a closed list, with a stated reason (Section 2.4). Still: quiescence's interior is a fixpoint at the leaf, move ordering is modeled by its load-bearing consequences only, and **no chess rules are formalized at all**. A reader should trust the theorems as statements about the search mechanisms, quantified over all abstract games, not as end-to-end verification of the Python.

Two things would narrow the gap, and we name them because they are the obvious next steps rather than because we have done them. The first is to verify `gen_moves` against the rules of chess — currently assigned to a separate track that works on the Python semantics directly, and the one item on the development's open list. The second is cheaper and, we suspect, higher-yield per unit effort: instantiate the abstract `Game` with a concrete Lean chess instance and *differentially test* the Lean search against `sunfish.py` at fixed depth. That would not prove the correspondence, but it would convert it from an audited claim into an empirically exercised one, on the same footing as the 9,600-probe reference-versus-production battery that already backs Theorem 9.1.

Two author claims refuted by the proof assistant, mid-project. We record these because they are the strongest evidence in this paper that the machine was doing work the authors could not do by inspection. *(1) The fail-high-vs-V_{full} error.* While specifying LMR we claimed that fail-high reports remain sound against full negamax — the re-search guard, we reasoned, ensures no reduced fail high is ever yielded. Lean refused the induction: the guard protects only the *immediately* reduced move, while a parent fail high is certified by child fail lows, which are only sound against the children's *reduced* values — the contamination is recursive and cannot be scrubbed at one level. This is why the interval-shaped statement of Section 7 had to make V^{hi}'s children V^{lo}-valued, and why the mutually recursive pair existed at all — before both were deleted as guaranteeing nothing. *(2) The V_{reduced} target error.* We claimed the folklore fail-low target — reducible moves simply valued one ply shallower (negamaxShallow; deleted along with the interval-shaped statements it fed) — was the right lower target. Also false: on the re-search fall-through, the yielded value bounds the *deep* child value while the shallow value has just failed high and can exceed everything yielded; the sound per-move entry is the *minimum* of the two depths. Both errors would have survived code review; neither survived `lake build`.

Four more, from the second half. The pattern repeated, and the later instances are more embarrassing than the earlier ones because by then we knew to look. *(3) The reduced correction scan.* We proposed skipping quiescence-filtered moves in the legality scan — an obvious-looking optimization on a rare path — and wrote it. It is unsound, and the countermodel (`reducedScan_needs_premise`) is a legal child whose own legal moves all sit below its threshold, so the filtered value is exactly the sentinel. The proposal was reverted; full coverage was restored. *(4) The redundant killer lookup.* We argued from the code that reading `tp_move` before the virtual yields was a wasted dictionary access. Measurement refuted it: because the killer table is position-keyed and quiescence below the null move is not ply-limited, the null subtree can transpose back to the same board and store there — 10 disagreements in 3.0M reads. The lookup stayed where it was. *(5) The README's own driver claim.* Both this paper's source and the engine's comment asserted that MTD-bi only probes inside the mate band. Modeling the

driver refuted it for the carried first probe of a depth (`carried_gamma_escapes_band`). Note the direction of the fix: the *code* moved (the bracket widened) rather than the theorems weakening. *(6) A plausible chess fact.* "You cannot be in zugzwang while delivering forced mate" survived several reviews on plausibility alone, and is false (Section 10). We record it because it is the case that most clearly justifies the whole exercise: no amount of careful reading was going to catch it, and the only reason we looked was that the formalization forced the statement to be written down precisely enough to be searched for.

Measurement contamination. Every rule in `docs/TESTING.md` exists because its absence produced a confident wrong number in this repository: the −60 ELO phantom regression (shared checkout), the ±dozens swings from cross-game table persistence (process reuse), fictional error bars from deterministic engines without opening books, and the long-TC time bug invisible at fast TC. We treat the methodology document as part of the scientific apparatus, on the same footing as the proofs.

Threats. Single engine, one maintainer's conjectures; ELO error bars at 300 games are ±30-ish and several middle-block numbers in Table 5 are single runs — which in particular means the correctness campaign's "costs nothing" conclusion is a statement about a ±29 interval containing zero, not a demonstration of exact neutrality. The taxonomy of Section 7 is validated on one engine family (null-window MTD-bi) and its transfer to PVS/aspiration engines is argued, not proven. The invariant catalogue of Section 11 inherits every limit of the model, and we restate the largest one: the correspondence between the Lean definitions and the Python is audited and CI-guarded, not proved. Finally, the three refuted assumptions were found by targeted search over real positions; that method establishes falsity conclusively when it succeeds, but its failure to find a witness for a fourth assumption would not be evidence of that assumption's truth, and we have not treated it as such anywhere in this paper.

17 Availability

Everything is in the public sunfish repository [1]: the engine (`sunfish.py`), the Lean development (`formal/`, building with `lake build` from a pinned Lean 4.12.0 toolchain, zero dependencies beyond core Lean and the `omega` tactic — no Mathlib — so a full build takes seconds), the executable reference specification that Theorem 9.1 relates to the shipped loop, the drift guard (`formal/scripts/model_audit.py`, run by CI), the methodology (`docs/TESTING.md`), and the premise inventory, correspondence table and PR-guideline ledger (`formal/README.md`). The guideline is worth singling out for anyone extending the engine: it states, per mechanism, which theorem a change must preserve — which is the practical form the whole campaign takes once the paper is closed. CI builds the proofs and plays real protocol games: fastchess tournaments and a mock-Lichess integration test that drives the engine over the wire exactly as the production bot is driven. The engine itself plays continuously on Lichess as `sunfish-engine`.

Acknowledgments.

References

[1] Thomas Dybdahl Ahle and contributors. Sunfish: a python chess engine in 131 lines of code. https://github.com/thomasahle/sunfish, 2026. Formal development in `formal/`; plays on Lichess as `sunfish-engine`.

[2] Disservin and contributors. fastchess: a command-line tool for managing chess engine tournaments. https://github.com/Disservin/fastchess, 2025.

[3] Ernst A. Heinz. Extended futility pruning. *ICCA Journal*, 21(2):75–83, 1998.

[4] Donald E. Knuth and Ronald W. Moore. An analysis of alpha-beta pruning. *Artificial Intelligence*, 6(4):293–326, 1975. doi:10.1016/0004-3702(75)90019-3.

[5] Formal verification of minimax algorithms. arXiv preprint, 2025. Verifies minimax variants with alpha-beta pruning and transposition tables in Dafny. https://arxiv.org/abs/2509.20138. TODO: fill in the author list from the arXiv page. arXiv:2509.20138.

[6] Tobias Nipkow. Alpha-beta pruning. *Archive of Formal Proofs*, 2024. https://isa-afp.org/entries/Alpha_Beta_Pruning.html, Formal proof development.

[7] Tobias Nipkow. Alpha-beta pruning verified. In *15th International Conference on Interactive Theorem Proving (ITP 2024)*, volume 309 of *LIPIcs*, pages 1:1–1:18. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2024. Invited talk. doi:10.4230/LIPIcs.ITP.2024.1.

[8] Aske Plaat, Jonathan Schaeffer, Wim Pijls, and Arie de Bruin. Best-first fixed-depth minimax algorithms. *Artificial Intelligence*, 87(1–2):255–293, 1996. doi:10.1016/0004-3702(95)00126-3.

Change	ELO	Disposition
Late Move Reductions (re-search)	$+49 \pm 34$	merged (LOS 99.8%, 300 games); later retired
TT store-clamp (crossing containment)	$+12 \pm 28$	merged (free); later a no-op
Deterministic LMR (first attempt)	-48 ± 33	declined
Deterministic LMR (conservative)	-16 ± 38	declined, then **merged** (`7f9f164`)
Sticky-entry consistency restoration	-33	declined
Ordering-only consistency restoration	-34	declined
Null-move soundness completion	-28	declined
Reverse futility	$+45 \to -1$	artifact (time regime); declined
Mate-score softening	$+19 \to -22$	artifact (time regime); declined
Removing deterministic LMR	$+69 \pm 40$, $+29 \pm 33$	**merged** (`7fdd741`); also $+19 \pm 45$ at 60+1
Re-search LMR, historical A/B replayed	$+41 \pm 36$	the `58883ea` +49 reproduces
Re-search LMR vs. no-LMR (today)	$+20 \pm 36$	the unsound variant's residual edge
Honest (min-semantics) LMR vs. no-LMR	0.00 ± 34	the edge was the bug; not merged
Verify-on-suspicion vs. prior master	-10.4 ± 28.7	**merged**; 300 games at 60+1
Stratified contract (deleting the depth-0 arm)	$+5.8 \pm 29.0$	**merged**; 300 games at 60+1
Removing the null move entirely	-34.9 ± 40.3	declined; 200 games at 60+1 — what the zugzwang bet buys

Table 5: Measured prices. Error bars are 95% where quoted; all matches per `docs/TESTING.md` (300 games, `tc=4+0.04`, opening book, adjudication) unless noted. The middle block prices *consistency*: every attempt to restore point-soundness under LMR — making the reduced search deterministic, keeping entries sticky, or restoring ordering only — appeared to cost 16–48 ELO. The proofs say what is being traded; these numbers say what the trade is worth. Dispositions reflect the full history: after the campaign, the maintainer deliberately accepted the cheapest restoration — conservative deterministic LMR at -16 ± 38 — in exchange for end-to-end point specs (Section 7.4); the re-search rows above the line are therefore "merged, later retired", and the store clamp became a provable no-op and was removed. The bottom block is the final campaign (Section 7.5): deterministic LMR's believed -16 was really ~ -50 — net negative — and with the honest variant worth exactly zero, all reductions were removed at `7fdd741`. The final block prices the correctness work of Sections 8–10: restoring soundness was free within measurement error, and the last row is the standing price of the one bet the engine still takes.